

列表

ADT: 循位置访问

03-A1

邓俊辉

deng@tsinghua.edu.cn

Don't lose the link. - Robin Milner

静态+动态 | 物理+逻辑

❖ 数据结构的操作林林总总，纵观无非三类：

- **静态**操作：元素的**集合**及元素之间的**结构**保持不变

可能修改元素的值，比如traverse；也可能不修改，比如search

- **动态**操作：元素之间的结构有所**调整**，比如sort；或者

元素会有所**增减**，比如insert、remove

❖ 向量**擅长**于前者，但对后者却显得**吃力**

究其根源，在于**循序访问**：将元素的**物理地址**与其**逻辑次序**绑定 //成于斯，败亦于斯

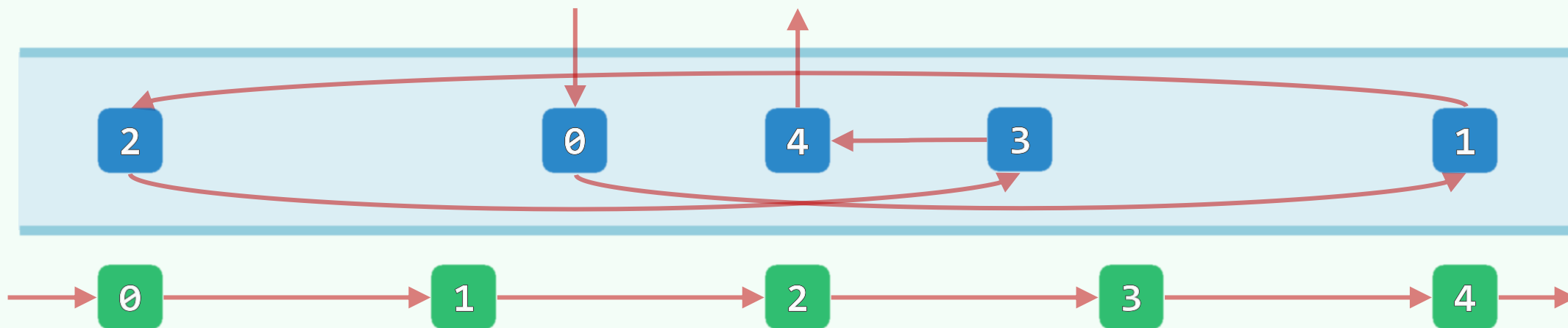
❖ 为实现高效的动态操作，或许...应该...

首先...将二者**解耦**... //不破不立

解耦

❖ **列表** | list: 是向量的姊妹，都来自于**序列** | sequence 家族

- 元素仍是依照**线性**次序排列，逻辑上也各自对应于一个**秩**，相邻元素亦互称**前驱**、**后继**
- 故依然记作 $\mathcal{L} = \{ a_0, a_1, a_2, \dots, a_{n-1} \}$



❖ 与向量不同，列表中各元素的物理次序**不再**与其逻辑次序直接相关

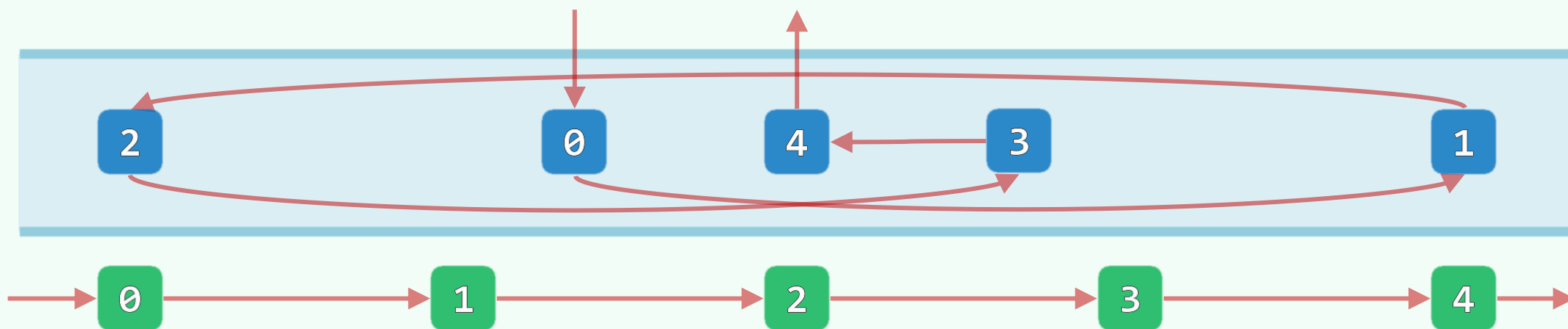
逻辑上紧邻的一对前驱与后继元素，物理上既**不必**保持前后次序，更**未必**相互紧邻

位置

❖ 为保证能够访问到每个元素

每一对逻辑上的紧邻元素，都需要彼此记录指向对方的**链接**|link，即**物理位置**

将**链接**抽象为元素的**位置**|position，我们便可以...



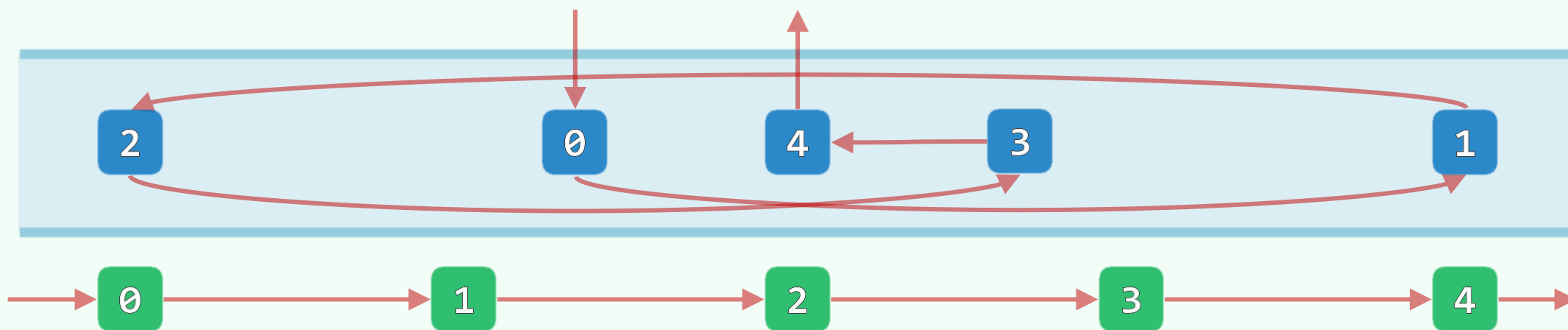
❖ ...循着位置，在**常数**时间内从一个元素转入下一个，从而可以抵达列表中的**任何**一个元素

犹如你借助**名片**，由某位朋友找到他的某位亲戚，进而他的同事、同事的战友、战友的同学、...

循位置访问

❖ Call-By-Position: 借助元素的位置顺藤摸瓜，找到任一目标元素，如此

- 物理位置得以**独立**于逻辑次序，动态操作仅会涉及**局部**的**常数**个元素而不再与**全局**相关
- 为实现高效率的动态操作提供了**机制**上的可能与保证

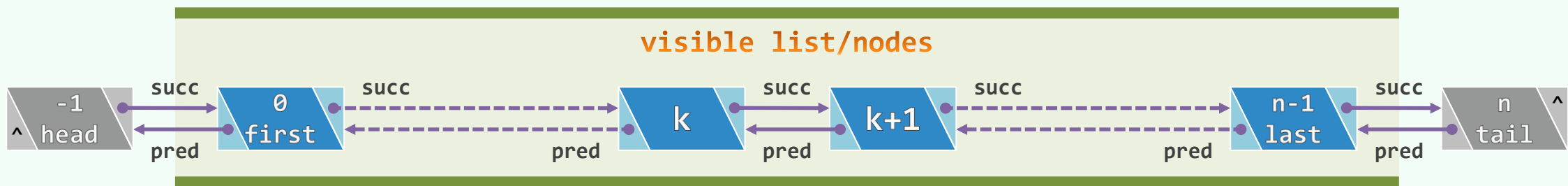


❖ 我们很快就会看到，列表的多数**动态**操作都可以在**常数**时间内完成！

如何具体实现呢？

链表+节点

- ❖ 链表|linked list: 列表的典型实现方式, 分两级封装而成
- ❖ 对应于列表的元素, 链表的基本单位为**节点**|node
 - 既封装了元素所对应的**数据项**: data
 - 也以指针的形式记录了**前驱**、**后继**节点的位置: pred|succ



- ❖ 于是, 节点之间可通过指针相互索引、顺次访问

而为了**引入**新节点或**删除**原有节点, 也只需在**局部**调整少量相关节点之间的指针 //有望 $O(1)$

列表

ADT：接口与实现

03-A2

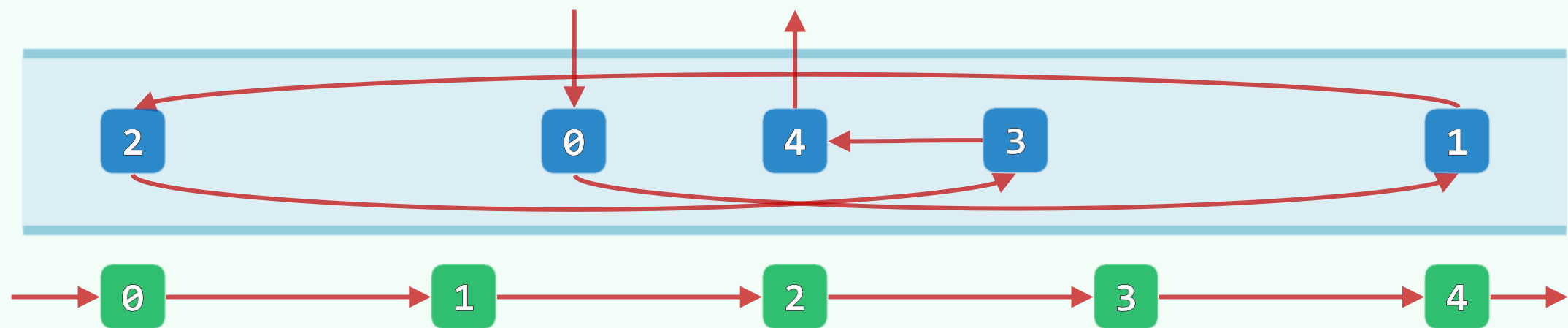
百隻駱駝繞山走，九十八隻在山後
尾駝露尾不見頭，頭駝露頭出山溝

邓俊辉

deng@tsinghua.edu.cn

ListNode ADT

操作接口	功能
data	节点所存数据对象
pred succ	前驱/后继节点的位置
insertPred(e) / insertSucc(e)	插入前驱/后继节点
movePred(s) / moveSucc(p)	将当前节点转移至紧靠于s之前 紧接于p之后



ListNode

```
template <typename T> using LNP = ListNode<T>*; //List Node Position
```

```
template <typename T> struct ListNode { //简洁起见, 完全开放而不再严格封装
```

```
    T data; LNP<T> pred, succ; //数值、前驱、后继
```

```
    ListNode(const T& e, LNP<T> p = NULL, LNP<T> s = NULL)
```

```
        : data(e), pred(p), succ(s) {} //默认构造器 (类T须已定义复制方法)
```

```
    LNP<T> insertPred( const T& e ); //前插入
```

```
    LNP<T> insertSucc( const T& e ); //后插入
```

```
    void movePred( LNP<T> s ); //转移至s之前驱
```

```
    void moveSucc( LNP<T> p ); //转移至p之后继
```

```
};
```



List ADT

操作接口	功能
size() empty()	报告节点总数 判定是否为空
first() last()	返回首 末节点的位置
insertFirst(e) insertLast(e)	将e当作首 末节点插入
insert(p, e) insert(e, p)	将e当作节点p的后继 前驱插入
remove(p)	删除节点p
sort()	排序
find(e) search(e)	在无序 有序列表中查找
dedup() unify()	无序 有序列表去重
traverse(visit())	遍历列表，统一按visit()处理所有节点

List

```
#include "listNode.h" //引入列表节点类
```

```
template <typename T> class List { //列表模板类
```

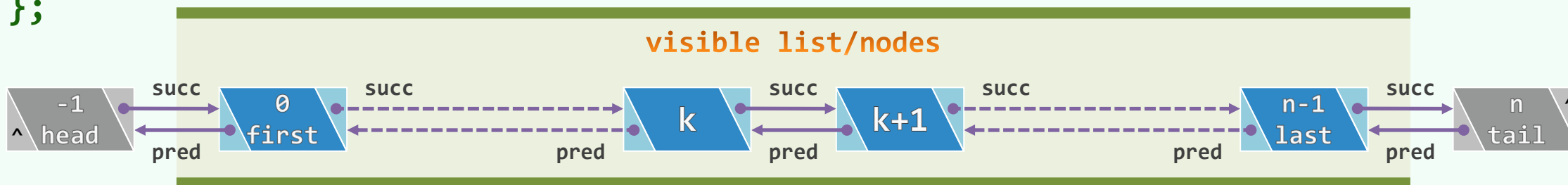
```
protected: Rank _size; LNP<T> head, tail; //哨兵
```

```
//头、首、末、尾节点的秩，可分别理解为-1、0、n-1、n
```

```
/* ... 内部函数 */
```

```
public: /* ... 构造函数、析构函数、只读接口、可写接口、遍历接口 */
```

```
};
```



List: 初始化

```
template <typename T> void List<T>::init() { //初始化, 创建列表对象时统一调用
```

```
    head = new ListNode<T>;
```

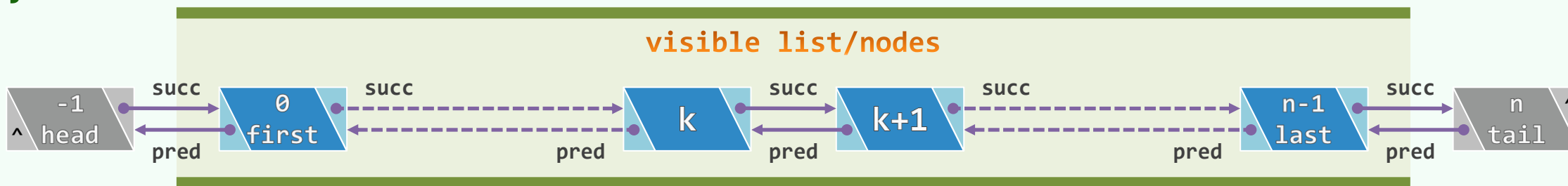
```
    tail = new ListNode<T>;
```

```
    head->succ = tail; head->pred = NULL;
```

```
    tail->pred = head; tail->succ = NULL;
```

```
    _size = 0;
```

```
}
```



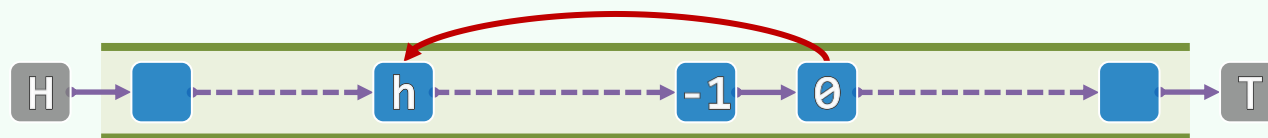
ListNode: 模仿循秩访问

❖ `template <typename T> LNP<T> List<T>::operator[] ( Rank r ) const`
`{ return *first() + r; }` //O(r)效率, 虽方便, 勿多用

❖ `template <typename T> LNP<T> ListNode<T>::operator+( Rank r )`
`for ( LNP<T> p = this; ; p = p->succ, r-- )` //从当前节点出发
`if ( 0 == r ) return p;` //顺数第r个节点即是目标节点
`}` //秩 == 前驱的总数



❖ `template <typename T> Rank ListNode<T>::operator- ( LNP<T> h ) {` //h<=this
`for ( Rank n = 0; ; h = h->succ, n++ )` //区间内的节点, 挨个报数
`if ( h == this ) return n;`
`}` //O(n)



列表

无序列表：插入、删除与移动

03-B1

邓俊辉

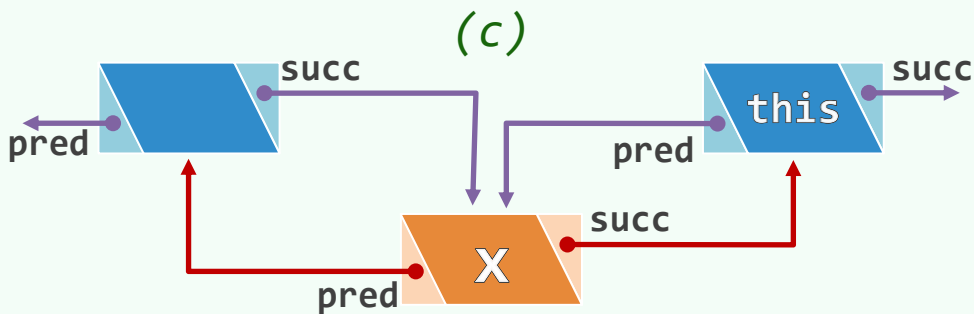
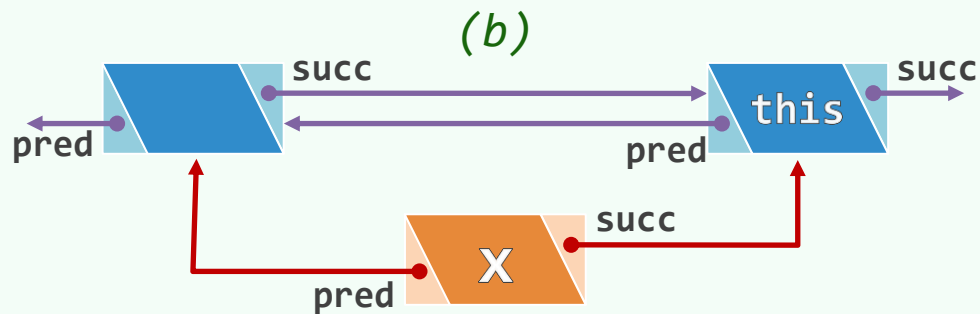
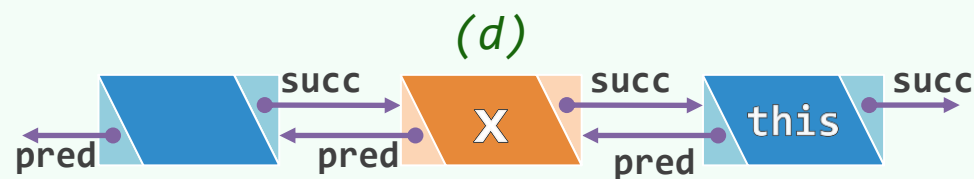
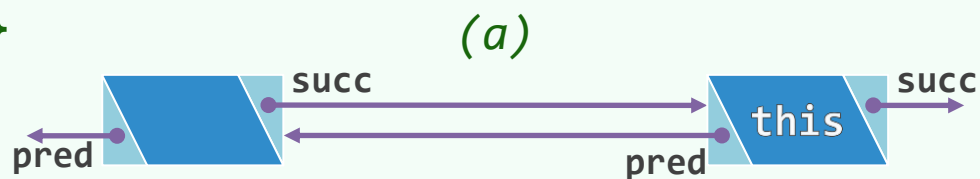
deng@tsinghua.edu.cn

List插入接口

- ❖ LNP<T> List<T>::insert(const T& e, LNP<T> p)
 { _size++; return p->insertPred(e); } //e当作p的**前驱**插入
- ❖ LNP<T> List<T>::insert(LNP<T> p, const T& e)
 { _size++; return p->insertPred(e); } //e当作p的**后继**插入
- ❖ LNP<T> List<T>::insertFirst(const T& e)
 { _size++; return head->insertSucc(e); } //e当作**首节点**插入
- ❖ LNP<T> List<T>::insertLast(const T& e)
 { _size++; return tail->insertPred(e); } //e当作**末节点**插入

ListNode插入算法

```
template <typename T> LNP<T> ListNode<T>::insertPred( const T & e ) {  
    LNP<T> x = new ListNode( e, pred, this ); //创建新节点 (pred或为head)  
    pred->succ = x; pred = x; return x; //列表接纳新节点 (操作次序不可颠倒)  
}
```



List删除算法

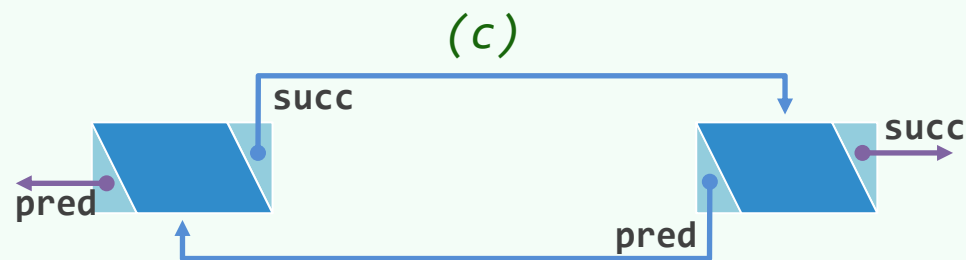
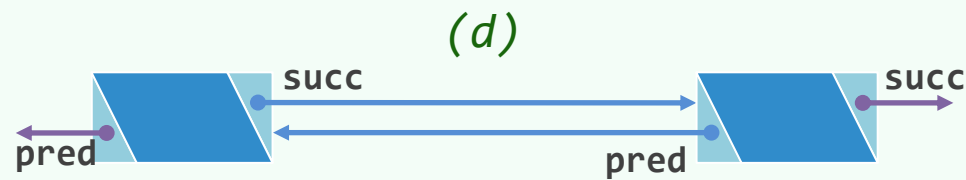
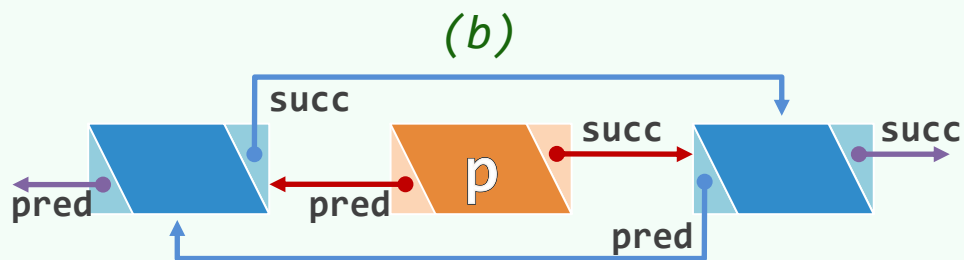
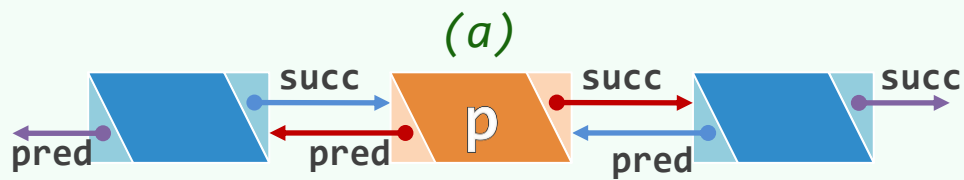
```
template <typename T> T List<T>::remove( LNP<T> p ) {
```

```
    T e = p->data; //备份数据项 (类型T可直接赋值)
```

```
    p->pred->succ = p->succ; p->succ->pred = p->pred; //pred|succ或为head|tail
```

```
    delete p; _size--; return e; //释放节点, 更新规模, 返回数据项
```

```
}
```



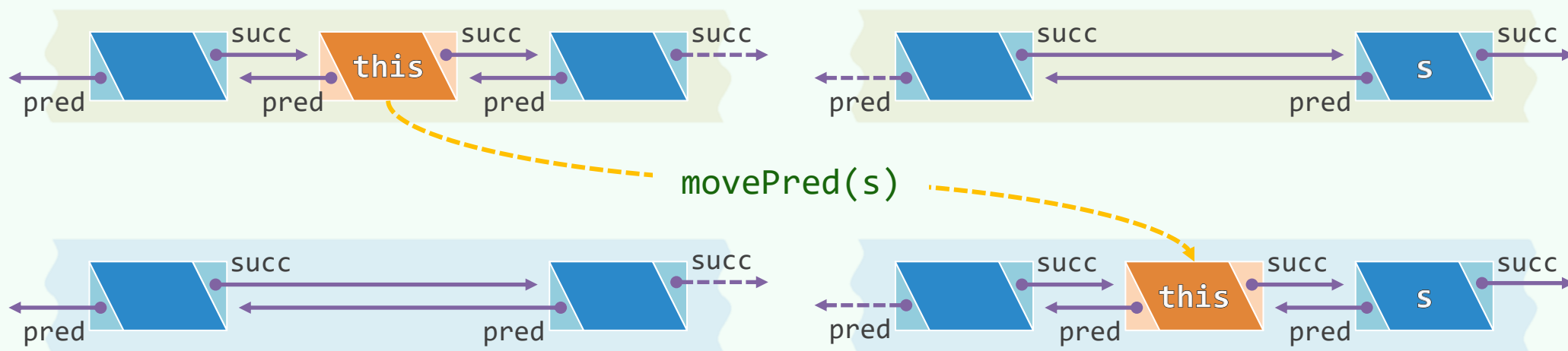
ListNode移动

```
template <typename T> void ListNode<T>::movePred( LNP<T> s ) { //O(1)
```

```
    pred->succ = succ; succ->pred = pred; //取出当前节点
```

```
    pred = s->pred; pred->succ = this; succ = s; s->pred = this; //转移至紧靠于s之前
```

```
} //moveSucc(p)完全对称
```



列表

无序列表：复制与析构

03-B2

“宇宙里有生有死.....爱情里也有死有生。”

“这是什么意思？” 剑云低声说，没有人回答他。

精神与我们的官能同生同长，同样菱黄：

哎呀！它一样要死亡

邓俊辉

deng@tsinghua.edu.cn

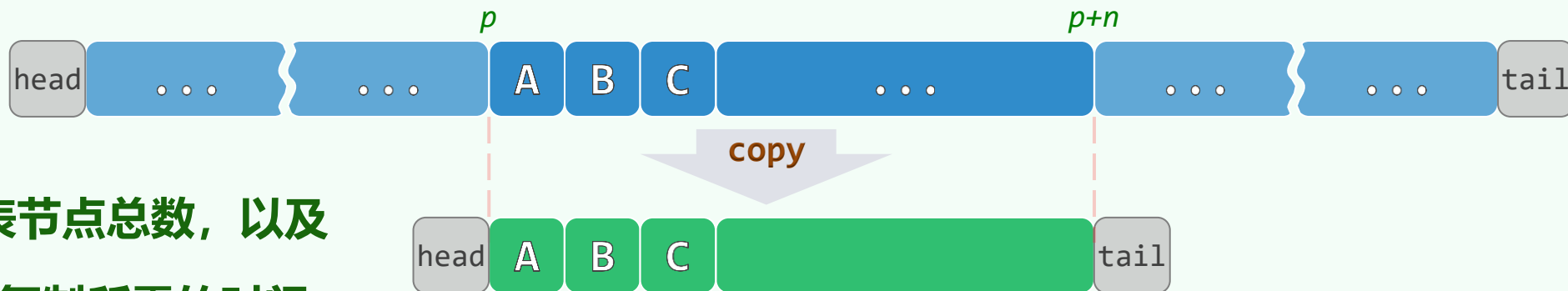
copy() | 深复制

❖ Deep Copy: 递归地复制**蓝本**中的每一层**嵌套**结构

```
❖ void List<T>::copy( LNP<T> p, Rank n ) { //依据蓝本区间, 复制出当前列表  
    for ( init(); n--; p = p->succ ) //先创建空表; 再将蓝本区间内各节点  
        insertLast( p->data ); //依次复制, 并作为末节点插入,  $O(1)$   
}
```

❖ 所需的时间

- 线性正比于列表节点总数, 以及
- 每个T对象的深复制所需的时间



```
❖ List<T>::List( List<T> const & L ) { copy( L.first(), L._size ); } //整体深复制
```

```
❖ List<T>::List( List<T> const & L, Rank r, Rank n ) { copy( L[r], n ); } //区段深复制
```

clean() | 析构

❖ 反过来，在析构时
也应递归地释放每一层的数据

❖ `template <typename T>`

```
List<T>::~~List() {
```

```
    for ( LNP<T> x = head;
```

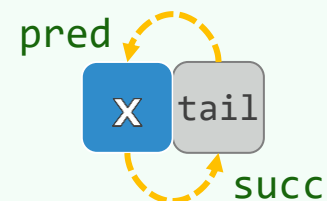
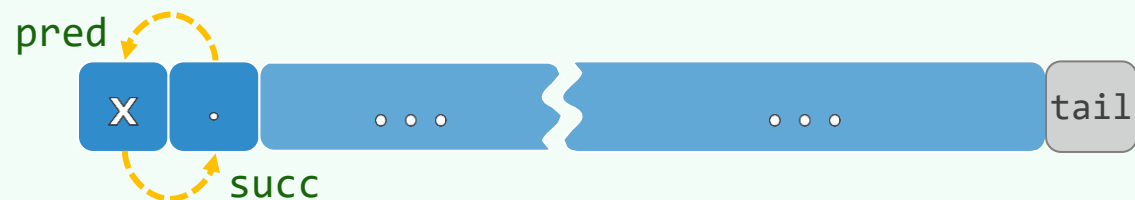
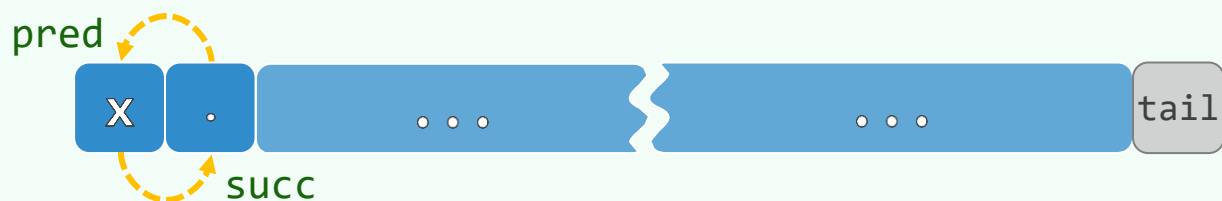
```
        x != tail;
```

```
        delete (x = x->succ)->pred );
```

```
    delete tail;
```

```
}
```

❖ 同理，所需的时间线性正比于列表节点总数，以及每个T对象的递归析构所需的时间



列表

无序列表：查找与去重

03-B3

顧長康噉甘蔗，先食尾。問所以，云：漸至佳境

有些事，你一辈子总也忘不掉。凡是让你揪心的事，在你身上，
都会发生两次。或两次以上

邓俊辉

deng@tsinghua.edu.cn

查找

```
template <typename T> LNP<T> //在无序列表区间(h,t)内, 找到等于e的最后者

List<T>::find( LNP<T> h, const T & e, LNP<T> t ) const { //h < t

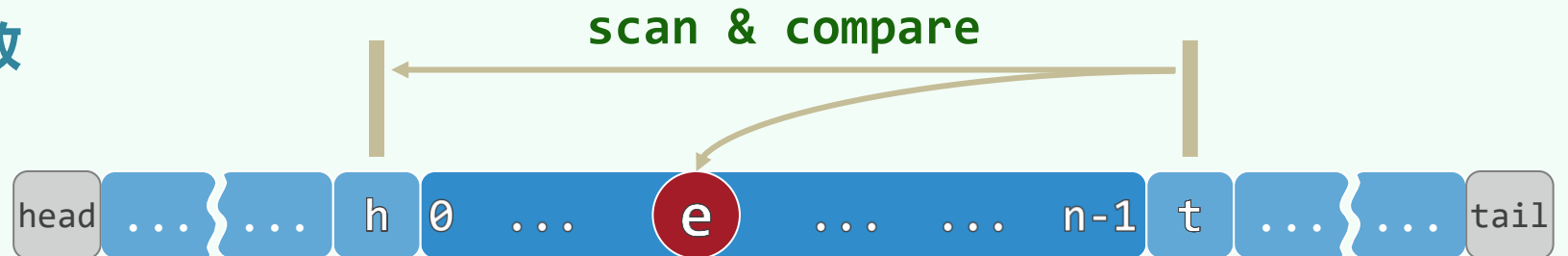
    while ( h != (t = t->pred) ) //自后向前

        if ( e == t->data ) //逐个比对 (假定类型T已重载 "==")

            return t; //直至命中

    return NULL; //或失败

} //find: O(n)
```



```
template <typename T>

LNP<T> find( const T & e ) const { return find( head, e, tail ); }
```

去重

```
❖ template <typename T> void List<T>::dedup() { //O(n^2)
    for ( LNP<T> p = first(); p != tail; p = p->succ ) //O(n)
        if ( LNP<T> q = find( head, p->data, p ) ) //O(n)
            remove ( q ); //O(1)
}
```

❖ **最坏情况** (比如**互异**) , $O(n^2)$

与Vector::dedup()一样



❖ **最好情况** (比如**全等**) , 只需 $O(n)$



得益于摆脱了**循序访问**的窠臼

列表

无序列表：遍历

03-B4

邓俊辉

deng@tsinghua.edu.cn

不要扯，等我一家家吃將來

traverse(): 函数指针/函数对象

❖ template <typename T>

```
void List<T>::traverse( void ( * visit )( T & ) ) {
```

```
    for ( LNP<T> p = head->succ; p != tail; p = p->succ )
```

```
        visit( p->data );
```

```
}
```



❖ template <typename T> template <typename VST>

```
void List<T>::traverse( VST & visit ) {
```

```
    for ( LNP<T> p = head->succ; p != tail; p = p->succ )
```

```
        visit( p->data );
```

```
}
```

实例：统一地将列表中的元素各自加一

❖ 先实现一个类（结构），可使单个T类型的元素加一

```
template <typename T> //假设T可直接递增或已重载操作符 “++”  
  
struct Increase //函数对象：通过重载操作符 “()” 实现  
  
    { virtual void operator()( T & e ) { e++; } }; //加一
```

❖ 再将其作为参数，传递给遍历算法

```
template <typename T> void increase( List<T> & L )  
  
    { L.traverse( Increase<T>() ); } //即可以之作为基本操作，遍历列表
```

❖ 课后练习：模仿此例，实现统一的减一、加倍、求和等更多的遍历功能

实例：统计元素总和，作为校验值

```
❖ template <typename T> struct Cumulate { //累计一个T类型对象的数值
    T & s;

    Cumulate( T & sum ) : s( sum ) {}

    virtual void operator()( T & e ) { s += e; } //约定T可直接相加
}; //Cumulate

❖ template <typename T> T sum( List<T> & L ) { //统计列表中所有元素的总和
    T sum = 0;

    L.traverse( Cumulate<T>( sum ) ); //以Cumulate为基本操作进行遍历

    return sum; //返回总和
} //sum
```

实例：统计逆序对总数，判断是否有序

❖ `template <typename T> struct Compare { //判断线性序列中的一个元素是否与其前驱顺序`

`T pred; int & inv; //前驱、（相邻）逆序对的计数器`

`Compare( int & inversions, T & first ) : pred( first ), inv( inversions ) {}`

`virtual void operator()( T & e ) { inv += ( pred > e ); pred = e; }`

`}; //Compare`

❖ `template <typename T> int inv( List<T> & L ) { //判断列表是否整体有序`

`int inversions = 0;`

`L.traverse( Compare<T>( inversions, L[0]->data ) ); //以Compare为基本操作做遍历`

`return inversions; //返回总数`

`} //inv`

列表

有序列表：去重与查找

昨夜
我梦见
你和我
说同一个字

古者詩三千餘篇，及至孔子，去其重，取可施於禮義

于是，他们急忙把自己的布袋卸在地上，各人打开自己的布袋。管家就搜查，从最大的开始，查到最小的。那杯竟在便雅悯的布袋里搜出来了

种种念起，无不出于找。离了现前，向外去找，根本让我们直觉麻木的是这个找

邓俊辉

deng@tsinghua.edu.cn

uniquify()

❖ 摒弃循秩访问转而循位置访问，使得动态操作只需常数时间，整体自然达到线性效率

❖ `template <typename T> void List<T>::uniquify() { //只需遍历列表一趟,  $O(n)$`

`if ( _size < 2 ) return; //过短的列表, 自然不含相等元素`

`for ( LNP<T> p = first(), q = p->succ; q != tail; q = p->succ )`

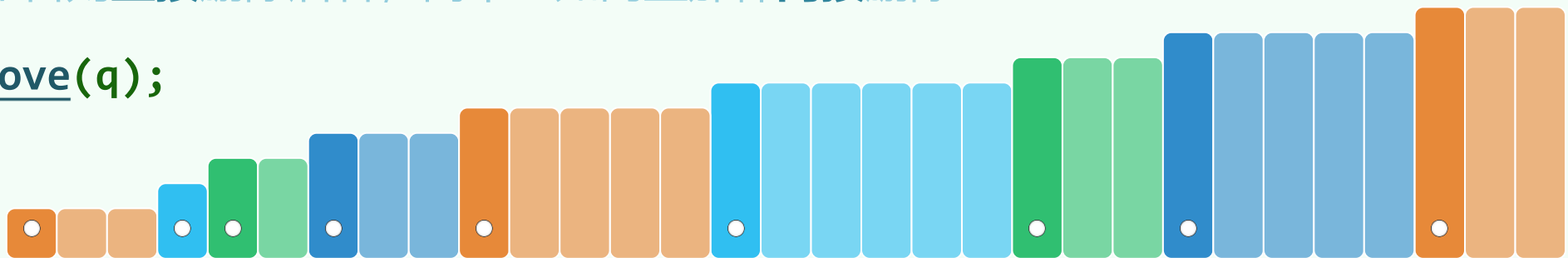
`if ( p->data != q->data ) //若p与其后继q不等`

`p = q; //则前进一步`

`else //否则直接删除后者, 而不必如向量那样间接删除`

`remove(q);`

`}`



search()

❖ 凡事一利一弊：动态操作性能提高的同时，静态操作性能可能反降

❖ `template <typename T> LNP<T> //有序列表区间(h,t):  $h < t$`

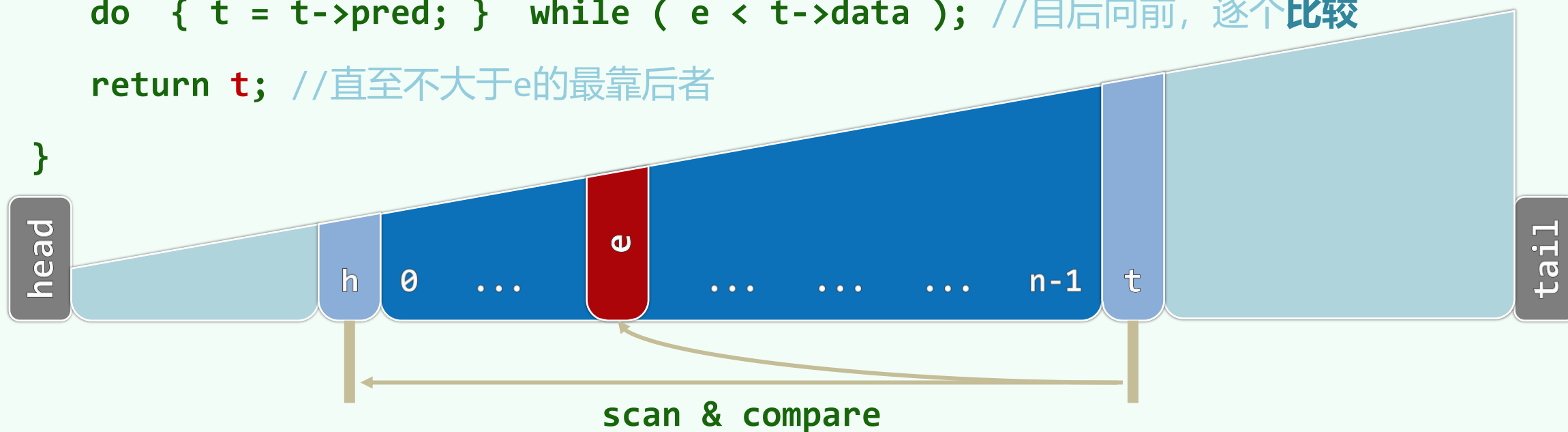
```
List<T>::search( LNP<T> h, const T & e, LNP<T> t ) const {
```

```
    if ( (h->succ == t) || (e < h->succ->data) ) return h; //越界情况, 先做特判
```

```
    do { t = t->pred; } while ( e < t->data ); //自后向前, 逐个比较
```

```
    return t; //直至不大于e的最靠后者
```

```
}
```





列表

选择排序

卡修斯永远讲道德，永远正经
他认为容忍恶棍的人自己就近于恶棍
只有在吃饭的时候——无疑他要选择
一个有鹿肉的坏蛋，而不要没肉的圣者

天下只有两种人。譬如一串葡萄到手，一种人挑最好的
先吃，另一种人把最好的留在最后吃

邓俊辉

deng@tsinghua.edu.cn

排序器：统一入口

```
template <typename T> void List<T>::sort( LNP<T> h, LNP<T> t ) {  
  
    switch ( rand() % 4 ) {  
  
        case 1 : insertionSort( h, t ); break; //插入排序  
  
        case 2 : selectionSort( h, t ); break; //选择排序  
  
        case 3 :      mergeSort( h, t ); break; //归并排序  
  
        default :      radixSort( h, t ); break; //希尔排序 (第14章)  
  
    } //随机选择算法, 以尽可能充分地测试; 应用时可视具体问题的特点, 灵活确定或扩充  
  
} //sort
```

起泡排序：温故知新

❖ 每趟扫描都需 $O(n)$ 次比较、 $O(n)$ 次交换

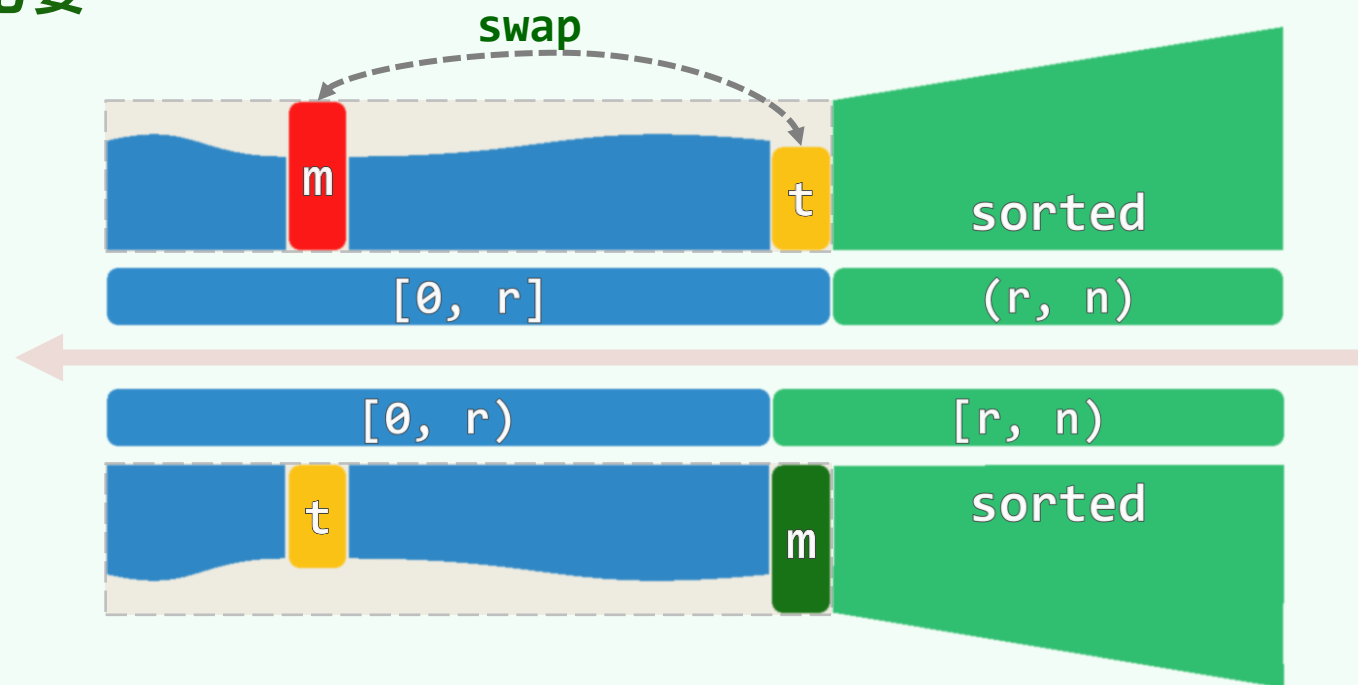
然而，其中的 $O(n)$ 次交换完全没有必要

❖ 每趟扫描的实质效果无非是

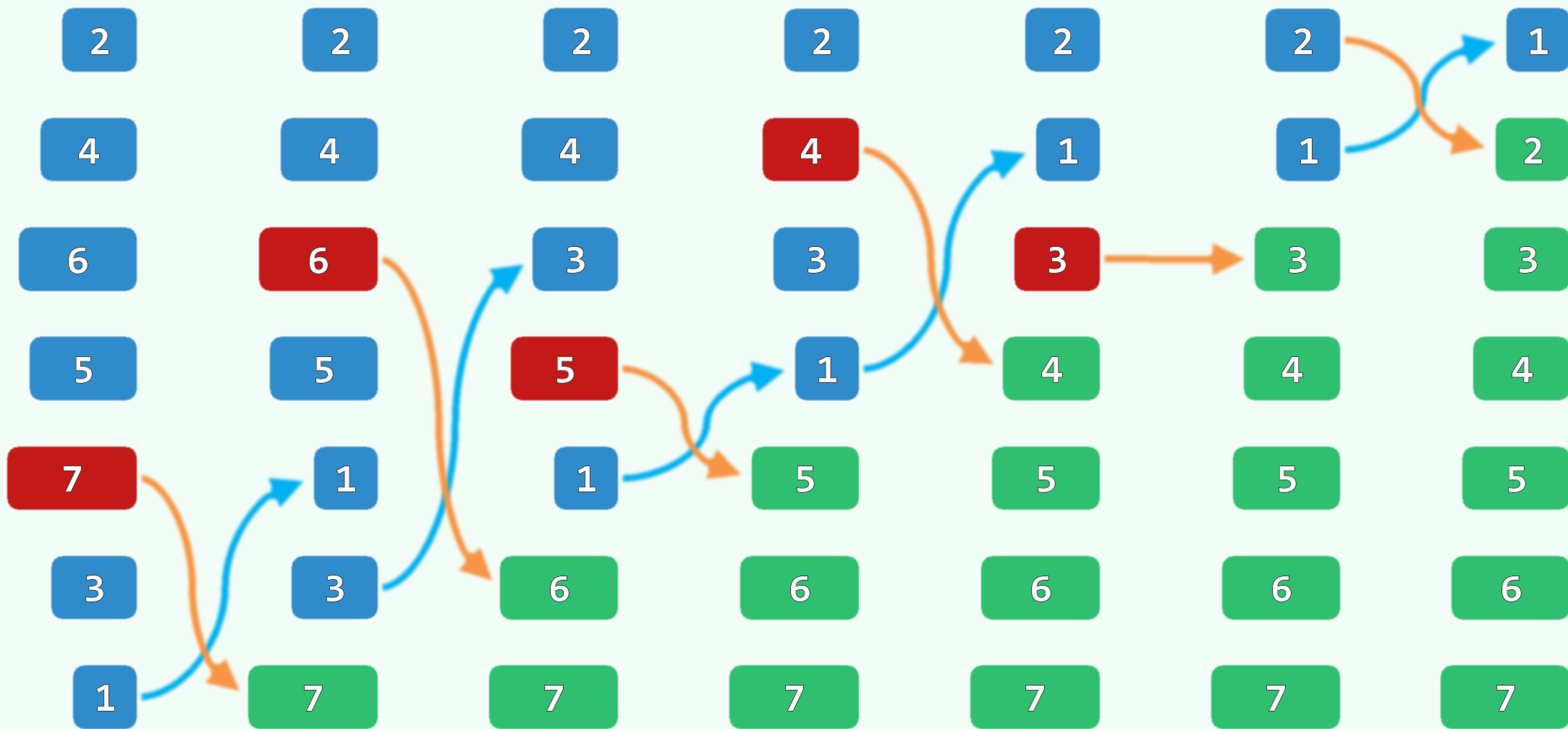
- 通过比较找到当前的最大元素 m ，并
- 通过交换使之就位

❖ 因此实际上，

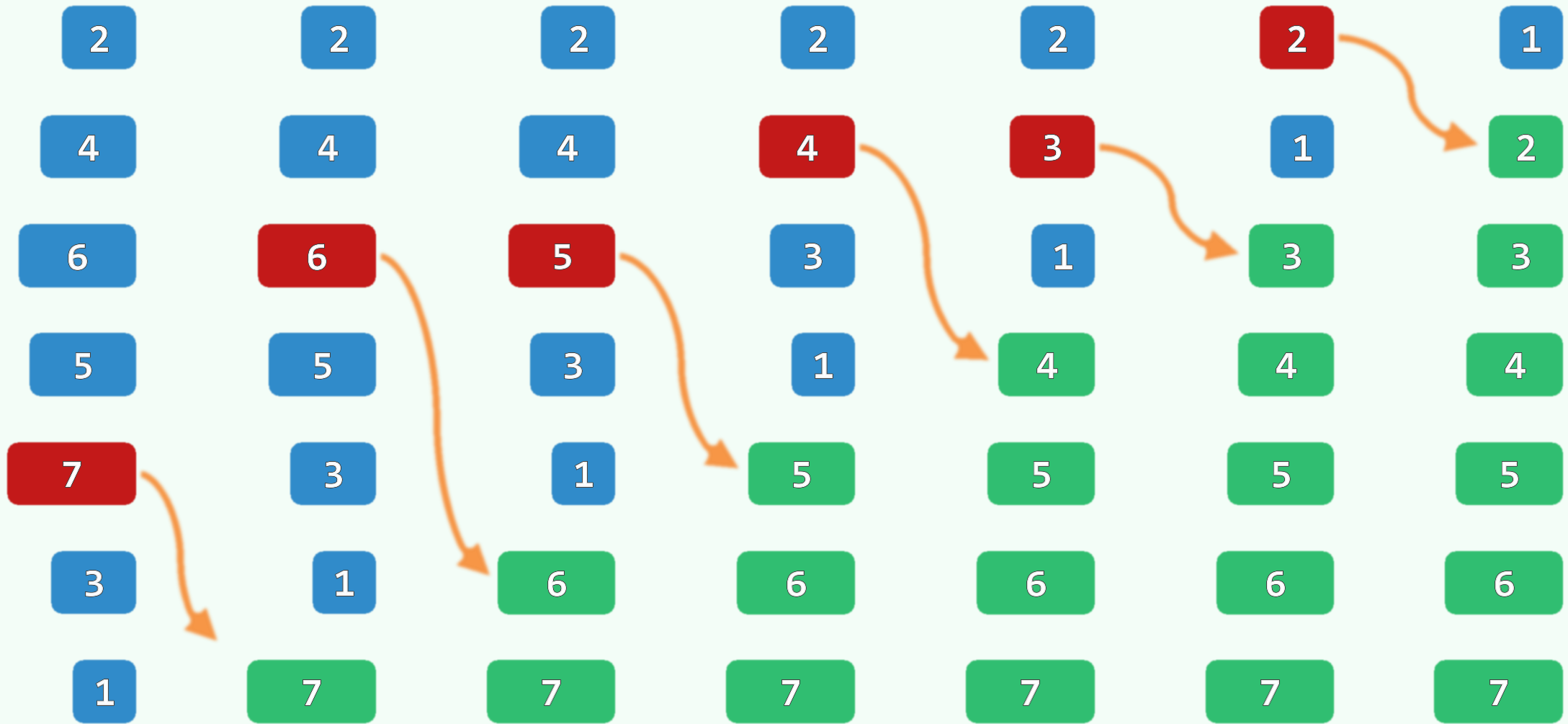
在经 $O(n)$ 次比较确定 m 之后，仅需一次交换即足矣



交换法



平移法



selectionSort()

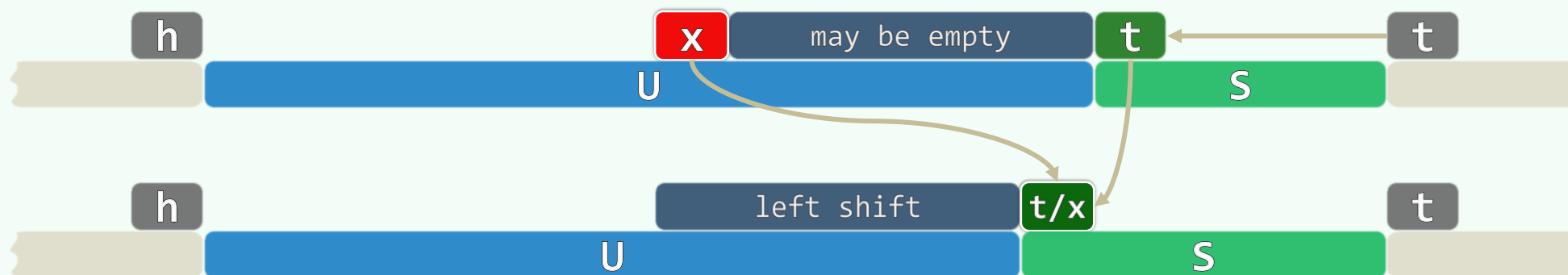
❖ `template <typename T> //h < t`

```
void List<T>::selectionSort( LNP<T> h, LNP<T> t ) {
```

```
    for ( ; h->succ != t; t = t->pred ) //反复从无序前缀(h,t)中
```

```
        getMax( h, t )->movePred( t ); //找出最大者, 并将其转至有序后缀[t,...)的前端
```

```
    } //selectionSort
```



getMax()

❖ template <typename T> //从区间(h,t)中选出最大者（若有多个，取**最靠后者**）：h < t

```
LNP<T> List<T>::getMax( LNP<T> h, LNP<T> t ) {
```

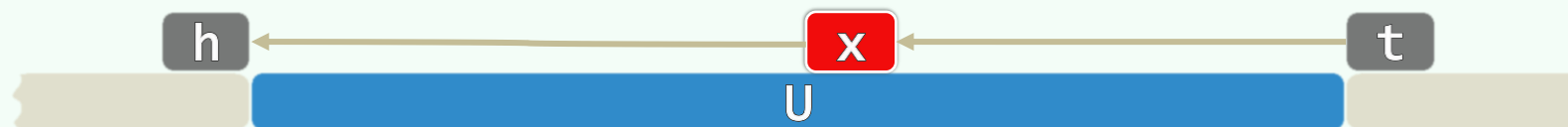
```
    for ( LNP<T> x = t = t->pred; x != h; x = x->pred ) //自后向前，逐个比较
```

```
        if ( t->data < x->data ) //见贤：优先级高低，取决于比较器
```

```
            t = x; //思齐
```

```
    return t; //最大者
```

```
} //getMax,  $O(n)$ 
```



稳定性：有**多个**元素同时命中时，约定返回其中**最靠后者**

❖ 回看`List<T>::getMax()`：这里是如何保证**后者优先**的？

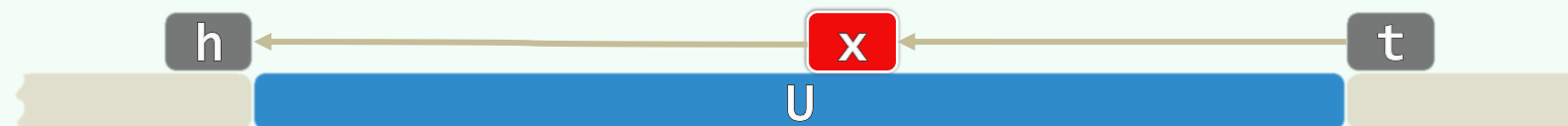
2 6a 4 6b 3 0 6c 1 5 7 8 9

2 6a 4 6b 3 0 1 5 6c 7 8 9

2 6a 4 3 0 1 5 6b 6c 7 8 9

2 4 3 0 1 5 6a 6b 6c 7 8 9

❖ 若采用平移法，如此即可保证**稳定性**：每一组**相等**的元素，都保持输入时的相对次序



性能分析

❖ 共 n 步迭代: movePred()累计 $O(n)$, getMax()累计 $O(n^2)$

对任何输入序列, getMax()累计耗时都是 $\Theta(n^2)$ ——没有**最好**情况!

❖ 实际上, $\Theta(n^2)$ 主要来自**比较**操作; **移动**操作只需 $\Theta(n)$ 次, 远少于起泡排序

就**常数**而言, **移动**操作远大于**比较**操作, 对**实际性能**影响更大

❖ 可否...每轮只做 $o(n)$ 次比较, 即找出当前的最大元素?

可以! ...利用高级数据结构, getMax()可改进至 $O(\log n)$

当然, 如此立即可以得到 $O(n \cdot \log n)$ 的排序算法

❖ 最大元素 m 有可能已经**就位**, 此时的movePred()显得**多余** <-- 可否避免这类**徒劳无益**的操作?

为此, 我们需要切换到一个**新**的视角, 以**重新**审视算法的过程...

列表

轮换

03-E

莫非你吃了我吩咐你不可吃的那树上的果子吗？

你所赐给我、与我同居的女人，她把那树上的果子给我，我就吃了

你作的是什么事呢？

那蛇引诱我，我就吃了

邓俊辉

deng@tsinghua.edu.cn

Permutation | Cycle

❖ 考查由 n 个互异元素构成的集合 $\mathcal{X}$ ，将其对应的**有序**序列记作 $S[0, n)$

❖ 由 $\mathcal{X}$ 排成的每个**序列** $A[0, n)$

都对应于 S 的一个**排列** $\tau()$:

对 $k \in [0, n)$ ，有 $S[\tau(k)] = A[k]$

k :	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S :	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
A :	J	N	P	M	A	I	G	O	D	C	H	B	K	L	F	E
$\tau(k)$:	9	13	15	12	0	8	6	14	3	2	7	1	10	11	5	4

❖ 若 $\tau^{(d)}(k) = k$ ，则 $\langle k = \tau^0(k), \tau^1(k), \tau^2(k), \tau^3(k), \dots, \tau^{(d-1)}(k) \rangle$ 构成一个长度为 d 的**轮换**

❖ 任何**排列**，都可

分解为若干**轮换**

0 9 2 15 4	1 13 11	3 12 10 7 14 5 8	6
A J C P E	B N L	D M K H O F I	G
J C P E A	N L B	M K H O F I D	G
9 2 15 4 0	13 11 1	12 10 7 14 5 8 3	6

采用交换法的选择排序

❖ 交换之前：m与t同属一个轮换c，且前后紧邻

❖ 交换之后：

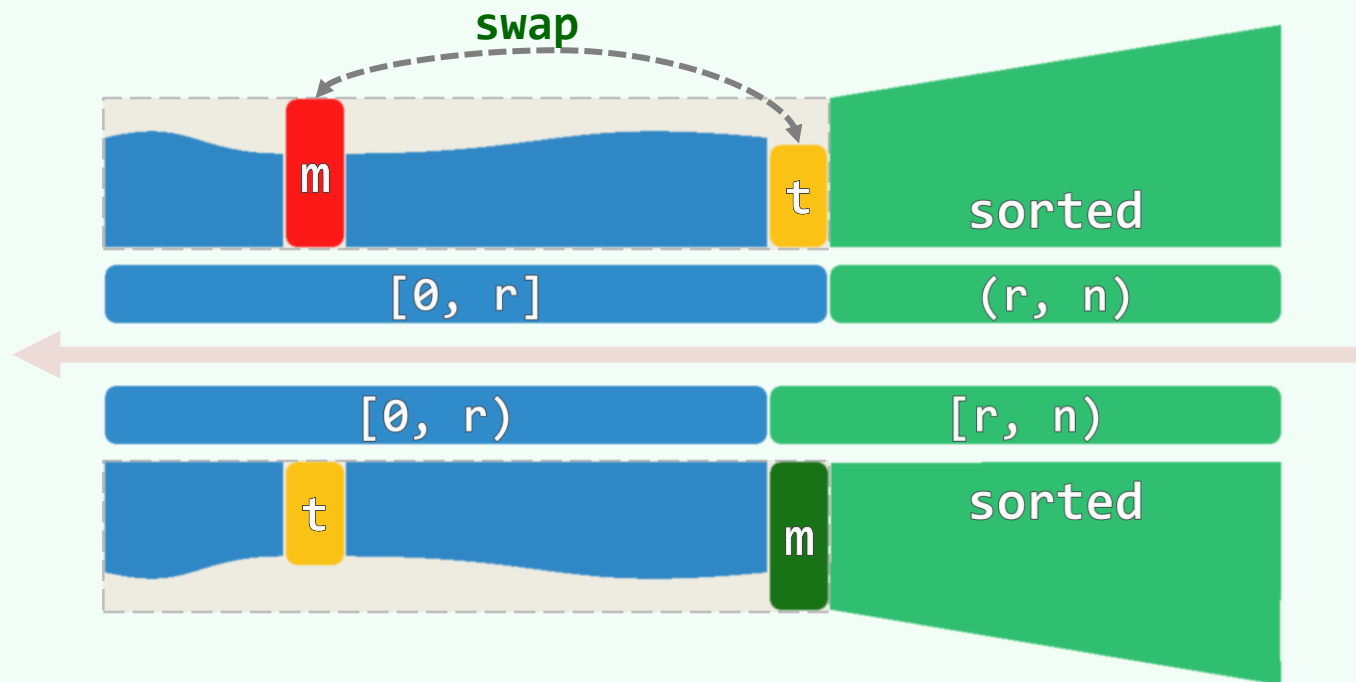
- m从c中逸出，并自成一个轮换
- c的长度减一（可能归零）
- 其余的轮换，保持不变

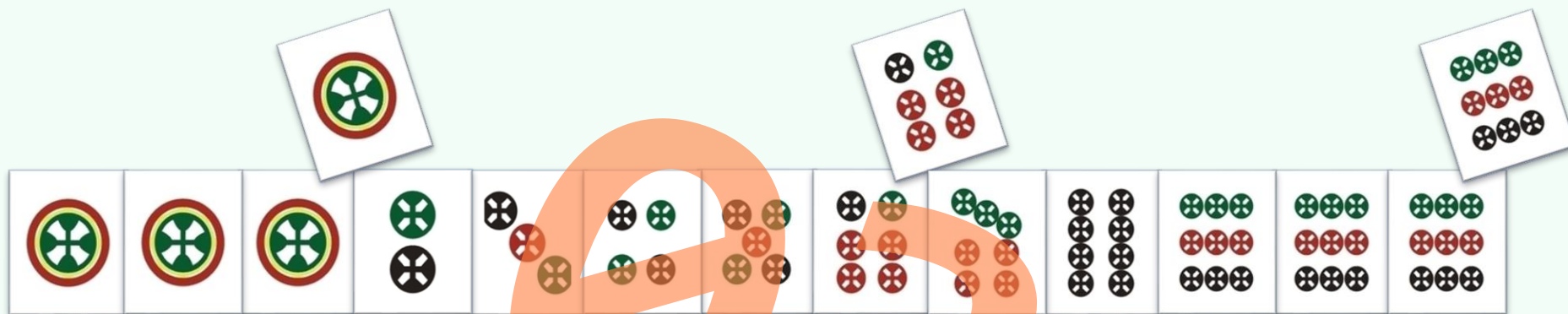
❖ c含k个轮换，就会出现k次多余的交换

$$- 1 \leq k \leq n$$

❖ 多余的交换 ~ c仅含m ~ m已经就位 ~ t在前缀中最大

$$- \mathbb{E}(k) = 1/1 + 1/2 + 1/3 + \cdots + 1/n = \Theta(\log n)$$





列表

插入排序

一語未了，只見寶玉笑嘻嘻的掬了一枝紅梅進來，眾丫鬢忙已接過，插入瓶內

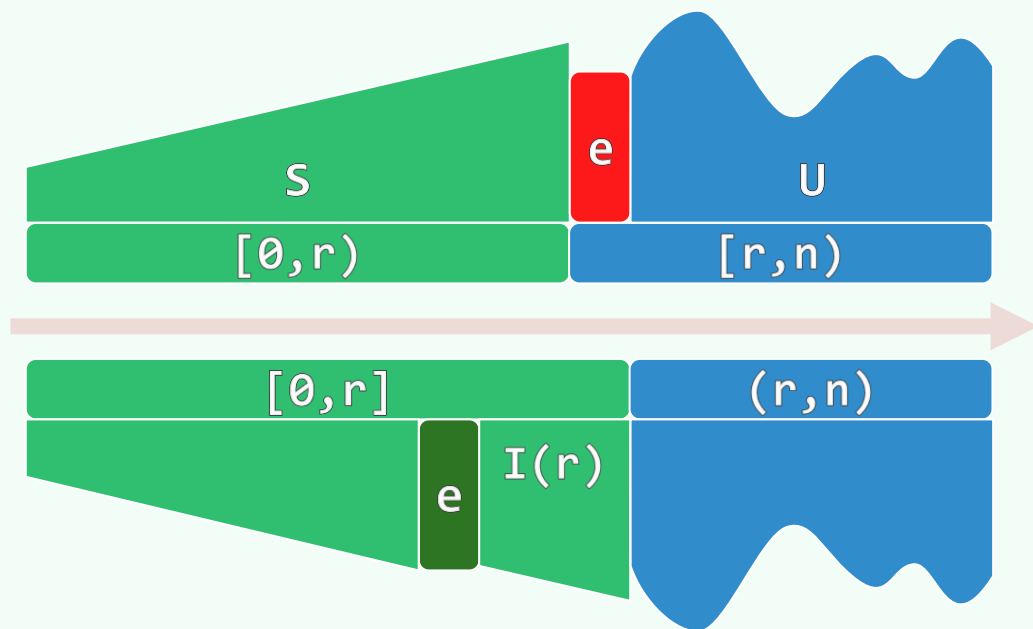
一日與樊、饒、章、蔣太太等看竹廿余周，余又負十餘元。後又看眾人打poker，覺無意味，二點始睡

人生如此美好，慢慢走，欣賞啊！

邓後輝

deng@tsinghua.edu.cn

减而治之



❖ 始终将序列视作两部分：

- 前缀 $S[0, r)$ ：有序
- 后缀 $U[r, n)$ ：待排序

❖ 初始化： $|S| = r = 0$

❖ 反复地，针对 $e = A[r]$

- 在 S 中查找适当位置
- 插入 e
- $r++$

实例

			5	2	7	4	6	3	1
iteration	sorted prefix	e	random suffix						
$\emptyset$	^	5	2	7	4	6	3	1	
1	5	2	7	4	6	3	1		
2	2 5	7	4	6	3	1			
3	2 5 7	4	6	3	1				
4	2 4 5 7	6	3	1					
5	2 4 5 6 7	3	1						
6	2 3 4 5 6 7	1							
7	1 2 3 4 5 6 7								

实现

```
template <typename T> //逐一引入节点r, 持续拓展有序前缀(h,r)

void List<T>::insertionSort( LNP<T> h, LNP<T> t ) {

    for ( LNP<T> r = h->succ, s = r->succ; r != t; r = s, s = r->succ )

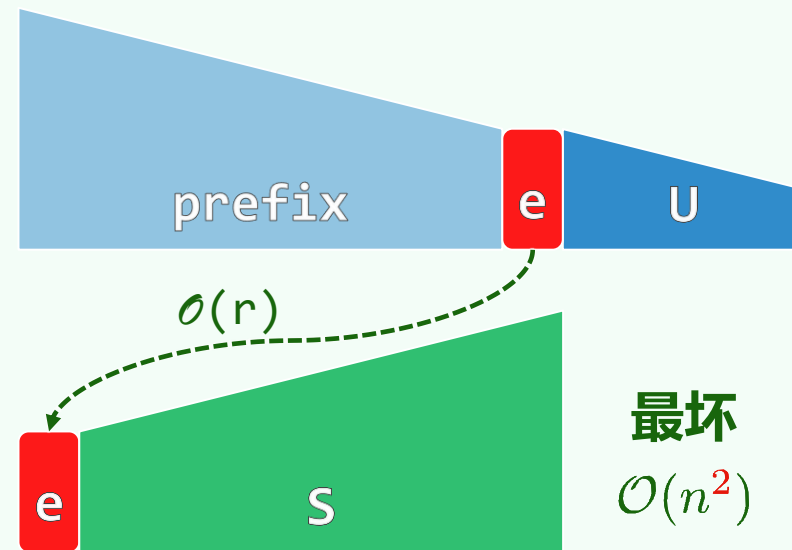
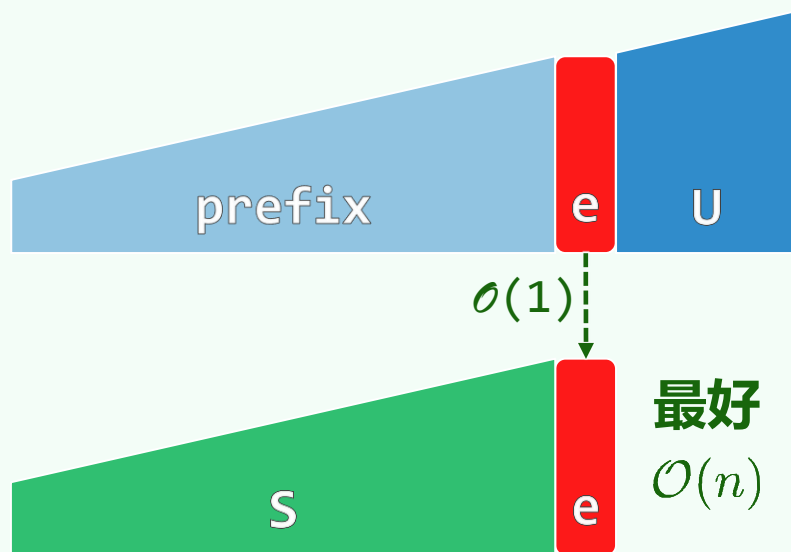
        r->moveSucc( search( h, r->data, r ) ); //为[r]在(h,r)内找到适当位置并插入

} //仅使用 $O(1)$ 辅助空间, 属于就地算法
```

❖ 得益于此前约定的search()接口语义, 前缀的确总是保持有序, 而且稳定

❖ 验证以下情况, 体会哨兵的作用: 前缀中有元素与r相等; r在前缀中最小/最大; 前缀为空; ...

复杂度



❖ 有实质的差异! Selectionsort呢?

❖ 输入**敏感性** | input-sensitivity

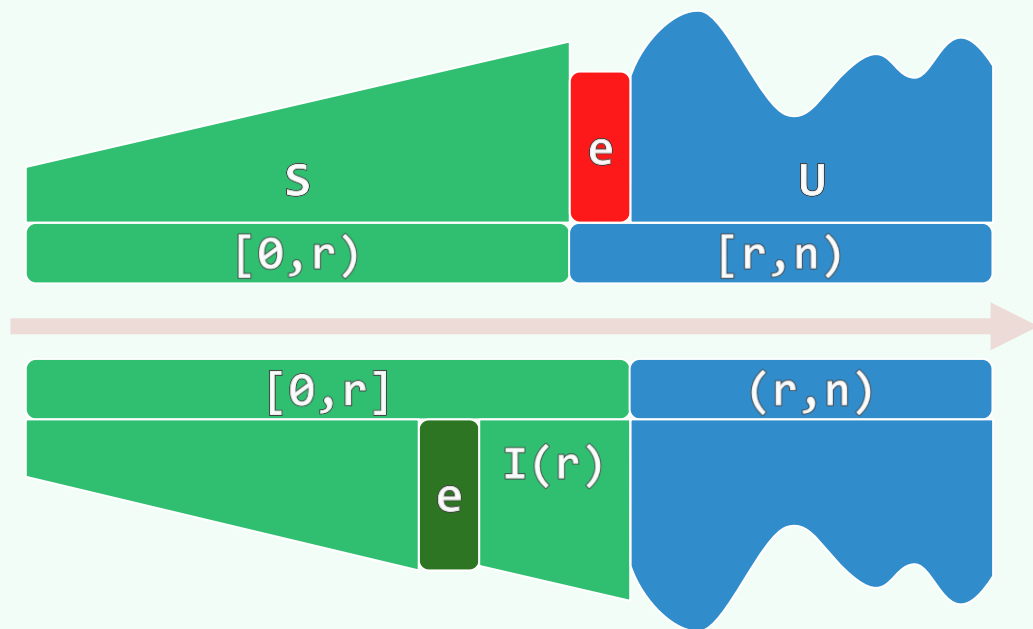
序列有多**乱**, 就花费多少时间

//稍后详解

❖ 为插入[r], 期望成本为 $r/2$

$$\text{总体} = \sum_{r=0}^{n-1} r/2 = O(n^2)$$

❖ 主要消耗来自查找, 为何不...改用...**binSearch()**?



❖ 日常牌戏中，为何**你我他**都用它？

❖ 改用**其他**算法...

发牌完毕，才能各自开始**理牌**

❖ Insertionsort

可以**边发边理**；**发完即理好**

❖ online

数据完全就绪之前，即可开始计算！

//何必一味追求**结果**，不妨充分享受**过程**

03-G

列表

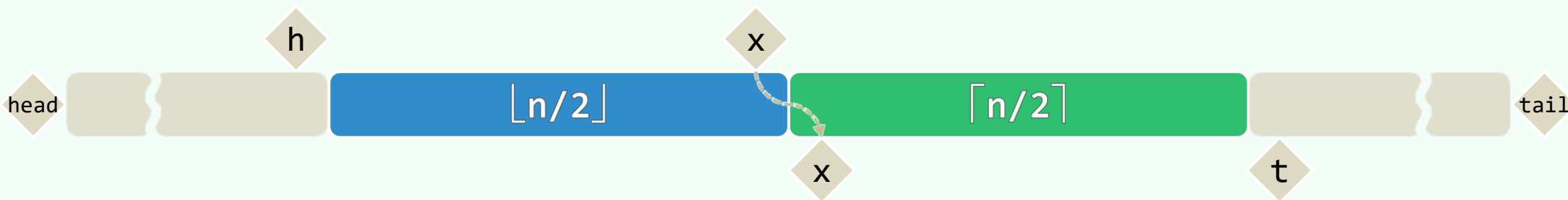
归并排序

日兩美其必合兮，孰信修而慕之
思九州之博大兮，豈惟是其有女

邓俊辉

deng@tsinghua.edu.cn

主算法



```
❖ template <typename T> void List<T>::mergeSort( LNP<T> h, LNP<T> t ) { //h < t

    Rank n = *t - h - 1; if ( n < 2 ) return; //区间长度不足2时，直接返回；否则...

    LNP<T> x = *h + n/2; //切分: (h,t) = (h,x] + (x,t)

    mergeSort( x, t ); mergeSort( h, x = x->succ ); //后缀、前缀各自排序

    merge( h, x, t ); //归并: (h,x) + [x,t)

}
```

//若归并可在线性时间内完成，则总体运行时间亦为 $\mathcal{O}(n \log n)$

二路归并

template <typename T> LNP<T> //将[y,t)中的节点, 分别转移至(h,y)中的适当位置

```
List<T>::merge( LNP<T> h, LNP<T> y, LNP<T> t ) {  
    for ( LNP<T> x = h->succ; (x != y) && (y != t); )
```

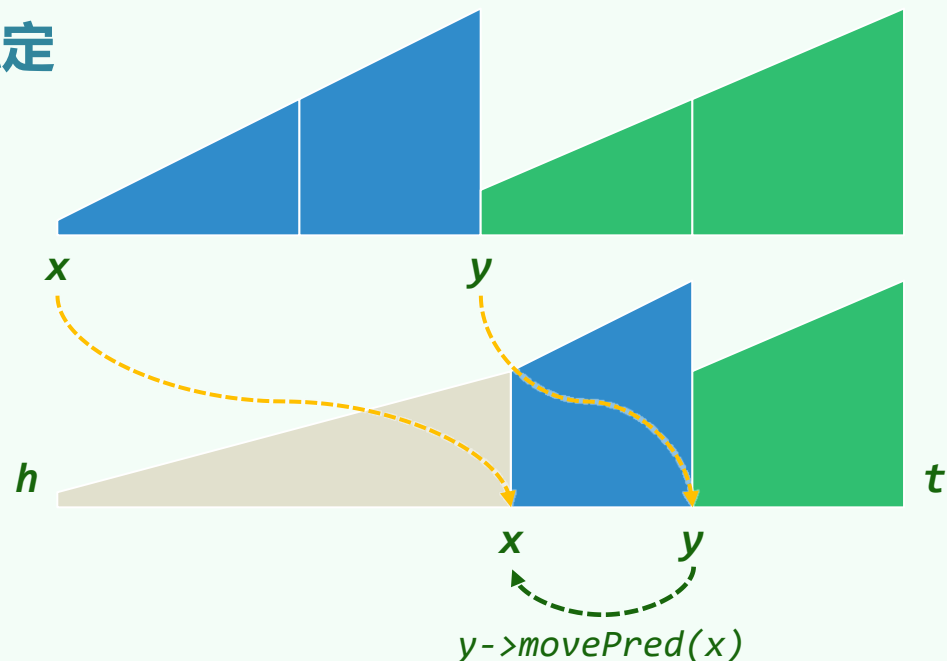
```
        if ( x->data > y->data ) //小者优先, 保证稳定
```

```
            (y = y->succ)->pred->movePred( x );
```

```
        else
```

```
            x = x->succ;
```

```
    } //O(n), 线性正比于节点总数
```



03-H

列表

逆序对

邓俊辉

deng@tsinghua.edu.cn

有象斯有對，對必反其為；有反斯有讎，讎必和而解

逆序对

❖ 在序列 $S[0, n)$ 中

如果有 $0 \leq i < j < n$ 且 $S[i] > S[j]$, 就称 $\langle i, j \rangle$ 为一个逆序对 | inversion

❖ 为便于统计, 可将逆序对统一记到后者的账上

$$\mathcal{I}(j) = \left| \left\{ 0 \leq i < j \mid S[i] > S[j] \right\} \right|$$

❖ 例: $\{ 5, 3, 1, 4, 2 \}$ 中, 共有 $0 + 1 + 2 + 1 + 3 = 7$ 个逆序对

$\{ 1, 2, 3, 4, 5 \}$ 中, 共有 $0 + 0 + 0 + 0 + 0 = 0$ 个逆序对

$\{ 5, 4, 3, 2, 1 \}$ 中, 共有 $0 + 1 + 2 + 3 + 4 = 10$ 个逆序对

❖ 显然, 逆序对总数 $\mathcal{I} = \sum_j \mathcal{I}(j) \leq \binom{n}{2} = \mathcal{O}(n^2)$

交换与修复

❖ **交换**一对逆序元素，逆序对**必然减少**

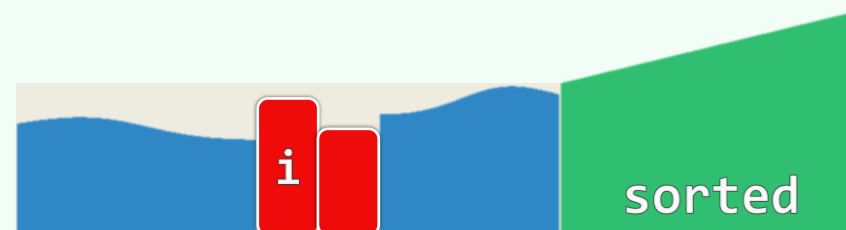
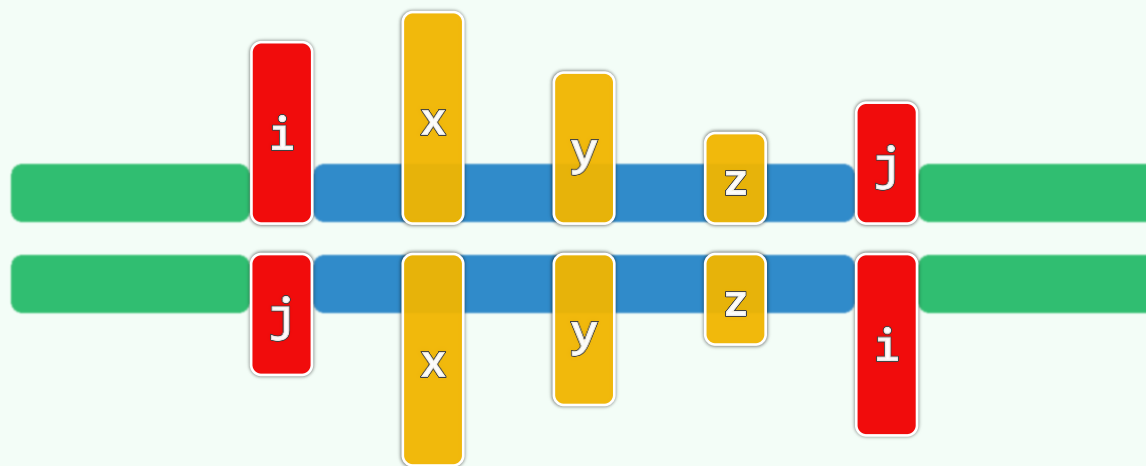
最少几个？最多呢？

❖ 通过**一次**交换，修复相距**更远**的逆序对
期望而言，可以**同时**修复**更多**的逆序对

❖ Bubble sort

- 每次交换的，都是**紧邻**的逆序对
- 逆序对只能**逐一**地修复

最坏情况下必然需要 $O(n^2)$ 时间



输入敏感

❖ 从逆序对的视角，重新审视插入排序...

❖ 针对 $e = S[r]$ 的那一步迭代

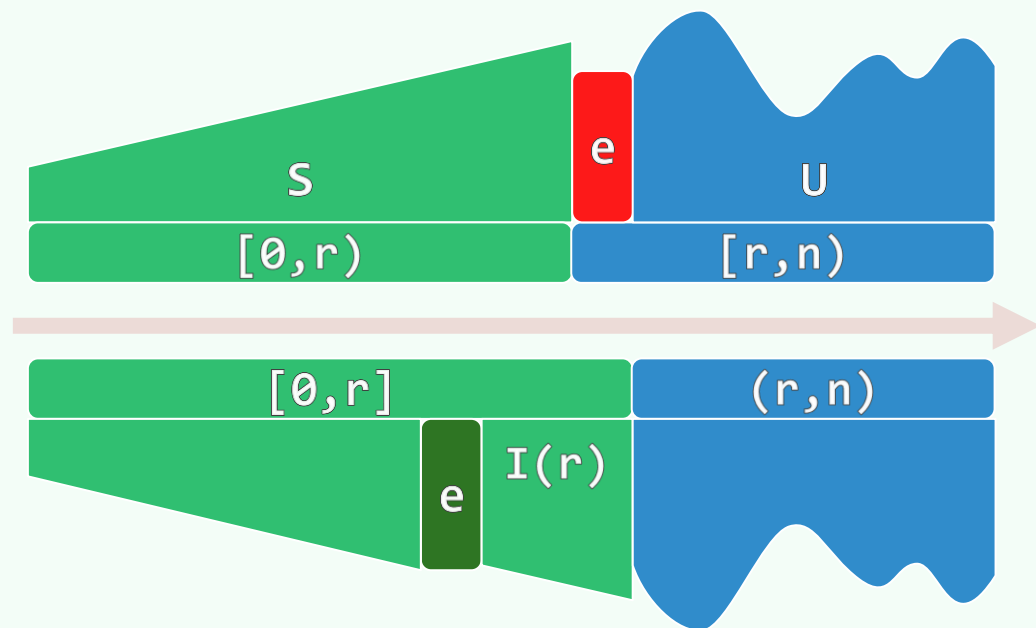
恰好需做 $\mathcal{I}(r)$ 次元素**比较**操作

❖ 若共含 $\mathcal{I}$ 个逆序对，则

- 比较的次数为 $\mathcal{O}(\mathcal{I})$
- 运行时间为 $\mathcal{O}(n + \mathcal{I})$

❖ 序列（含逆序对总数） ~ 衣物（脏污程度）

插入排序（所需时间） ~ 洗涤（用时长短）



计数：任意给定一个序列，如何统计其中逆序对的总数？

❖ 蛮力算法需要 $\Omega(n^2)$ 时间；而套用归并排序的框架，则仅需 $\mathcal{O}(n \log n)$ 时间...

❖ 分治： $L[lo, mi) + R[mi, hi)$

- 偏于一侧的逆序对可递归统计
- 实质地，只需考查跨界的逆序对

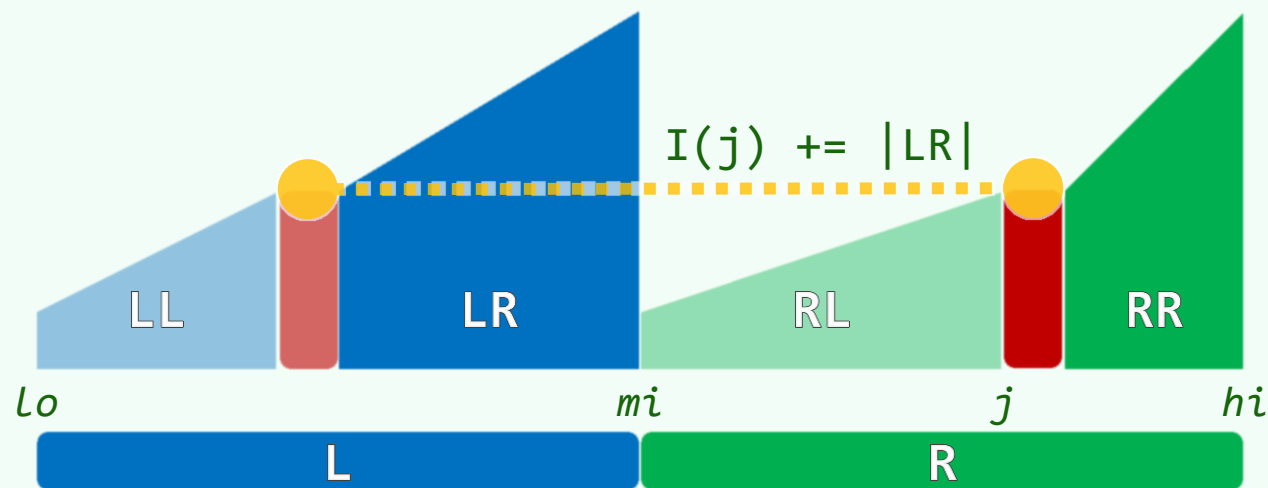
$$\langle i, j \rangle : lo \leq i < mi \leq j < hi$$

❖ 后一类逆序对，分别记到 $[j]$ 的账上：

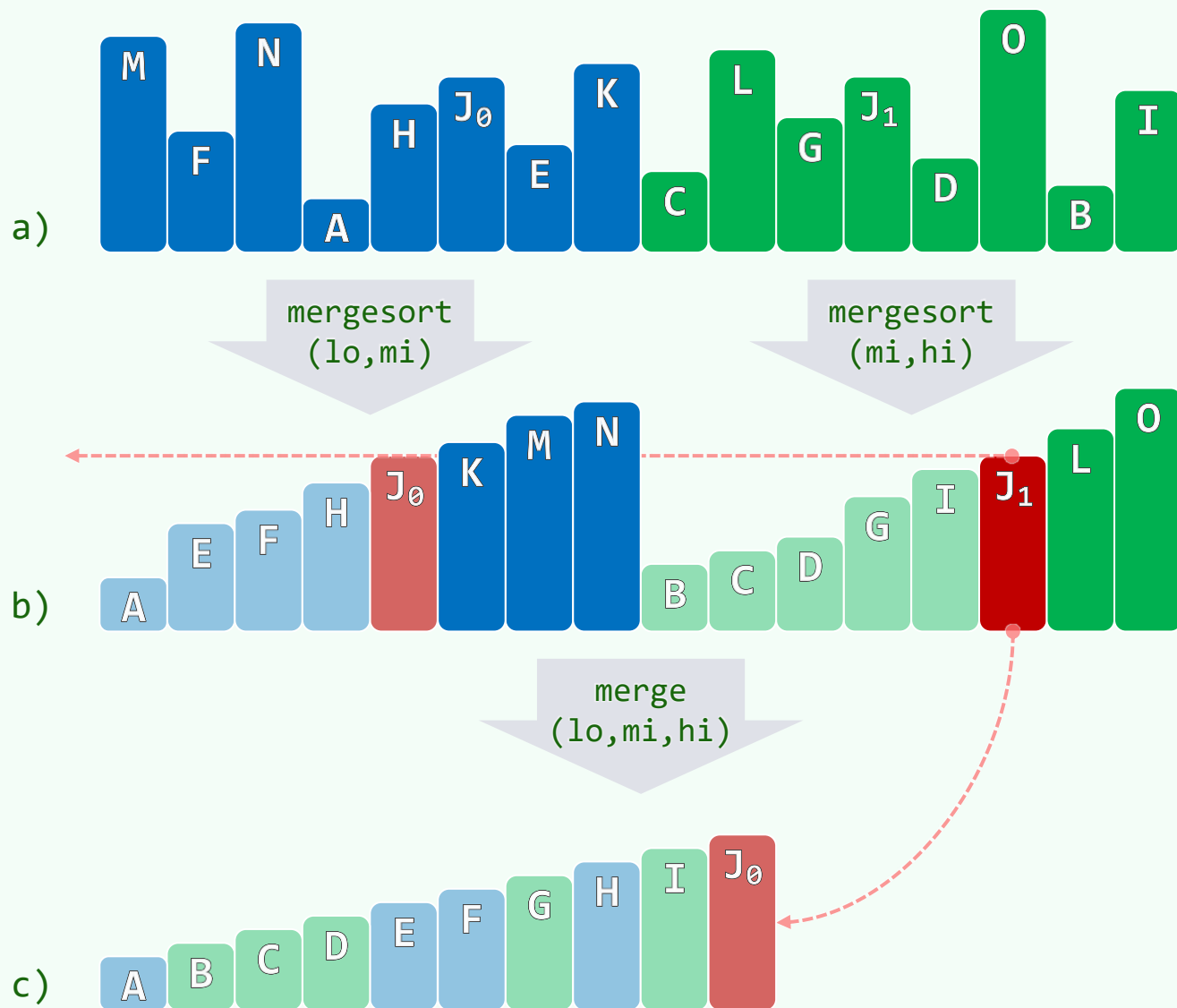
$$\hat{I}(j) = || \{ lo \leq i < mi \mid A[i] > A[j] \} ||$$

❖ 假想着对序列做归并排序，定格于 $[j]$ 刚被选中、即将出列的时刻

此时必有： $\hat{I}(j) = |LR|$



实例



紧邻的一对子任务/子序列：
若前缀中的MNK与后缀中的 J_1 逆序
则在子序列各自递归排序之后...

...归并进行至 J_1 **即将**出列就位时
前缀中必**恰好**只剩下KMN
虽次序或有调整，但总数不变

如此，便可统计出**跨越**前、后缀的逆序对
至于前、后缀各自**内部**的逆序对，则可...

03-XA

列表

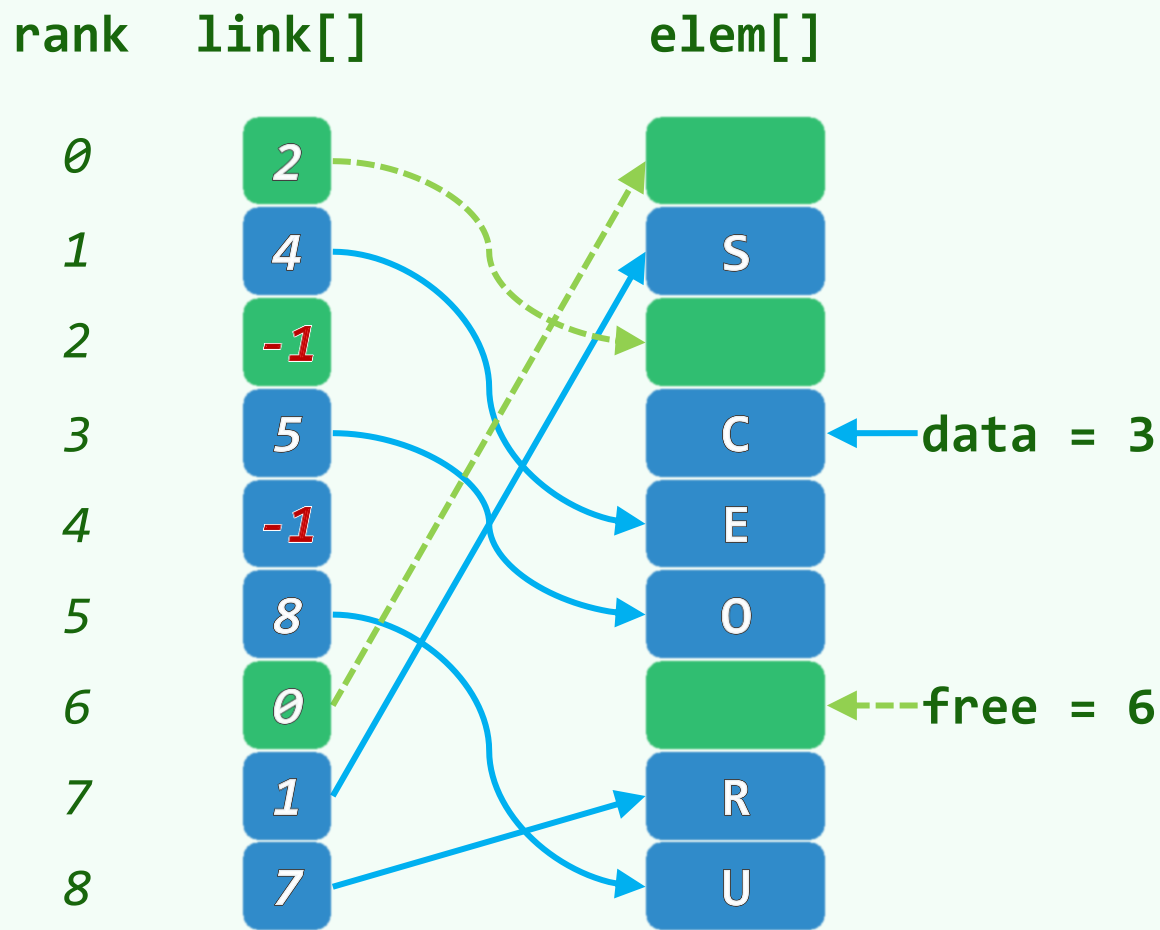
游标实现

邓俊辉

deng@tsinghua.edu.cn

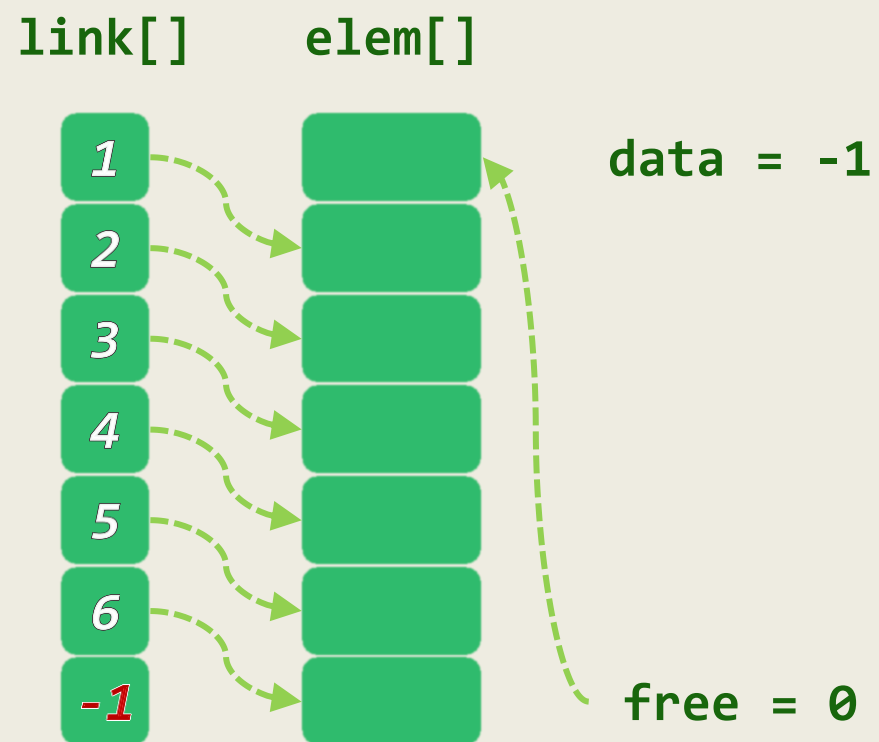
动机与构思

- ❖ 某些语言**不支持**指针类型，即便支持频繁的动态空间分配也**影响总体效率**
- ❖ 此时，又当如何实现列表结构呢？
- ❖ 利用线性数组，以游标方式模拟列表
 - elem[]: 对外可见的**数据项**
 - link[]: 数据项之间的**引用**
- ❖ 维护逻辑上**互补**的列表data和free
- ❖ 在插入或删除元素时，应如何调整？



实例 (1/4)

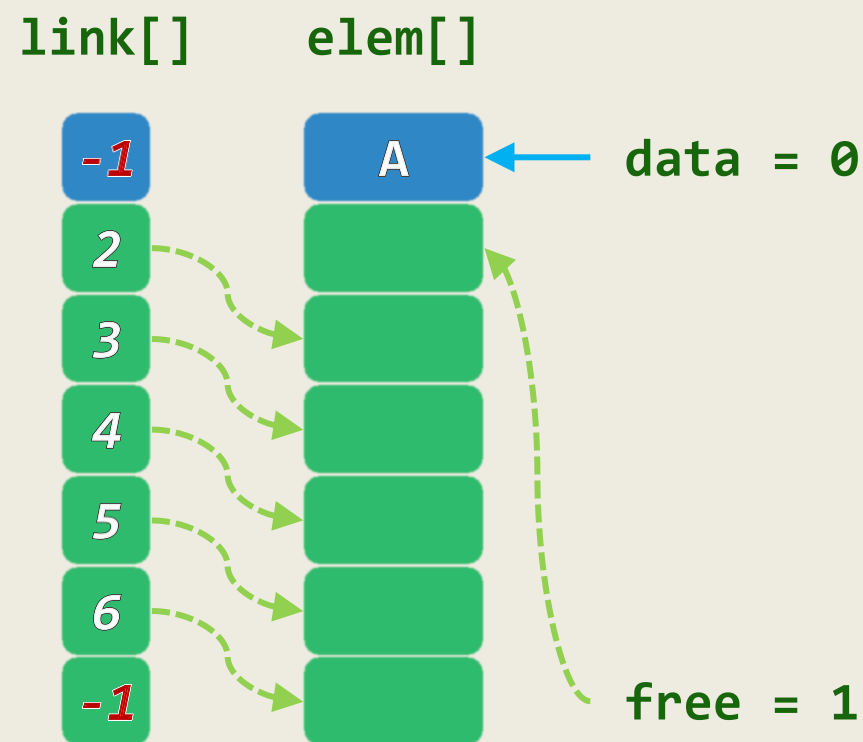
init(7)



rank

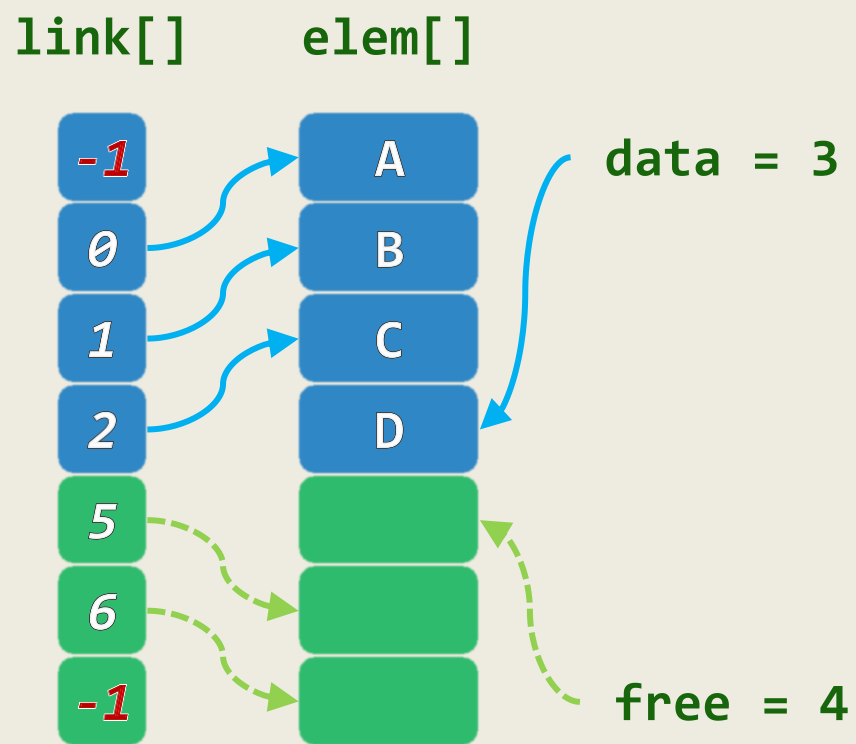
0
1
2
3
4
5
6

insert('A')



实例 (2/4)

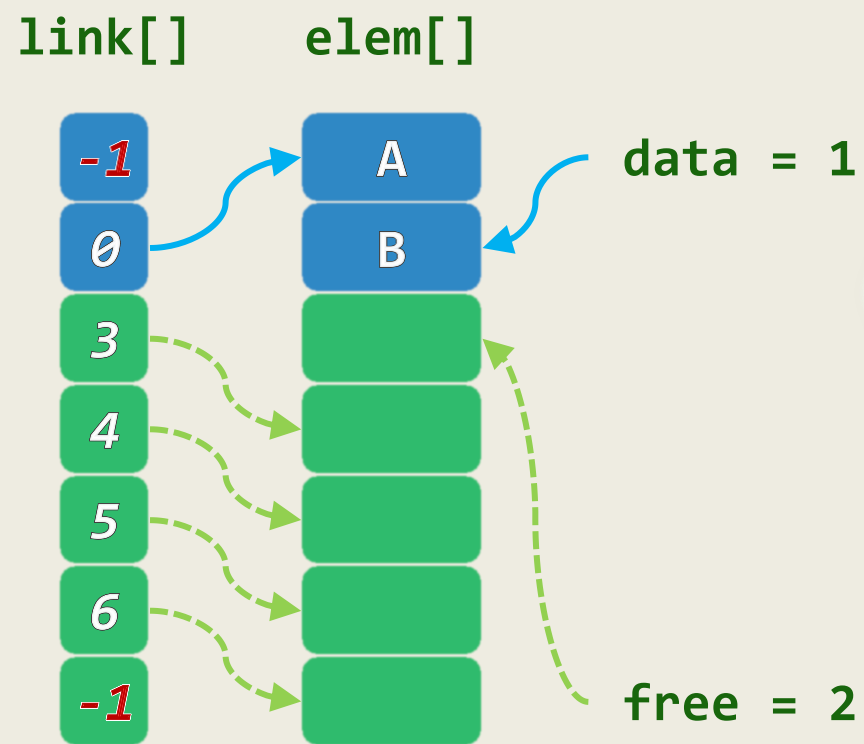
insert('C')
insert('D')



rank

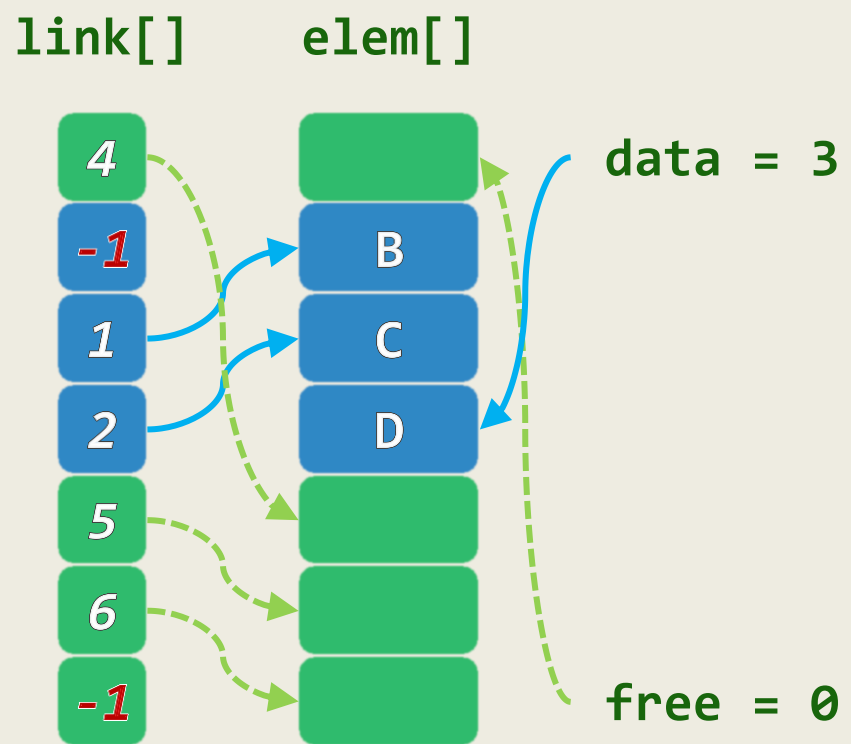
0
1
2
3
4
5
6

insert('B')

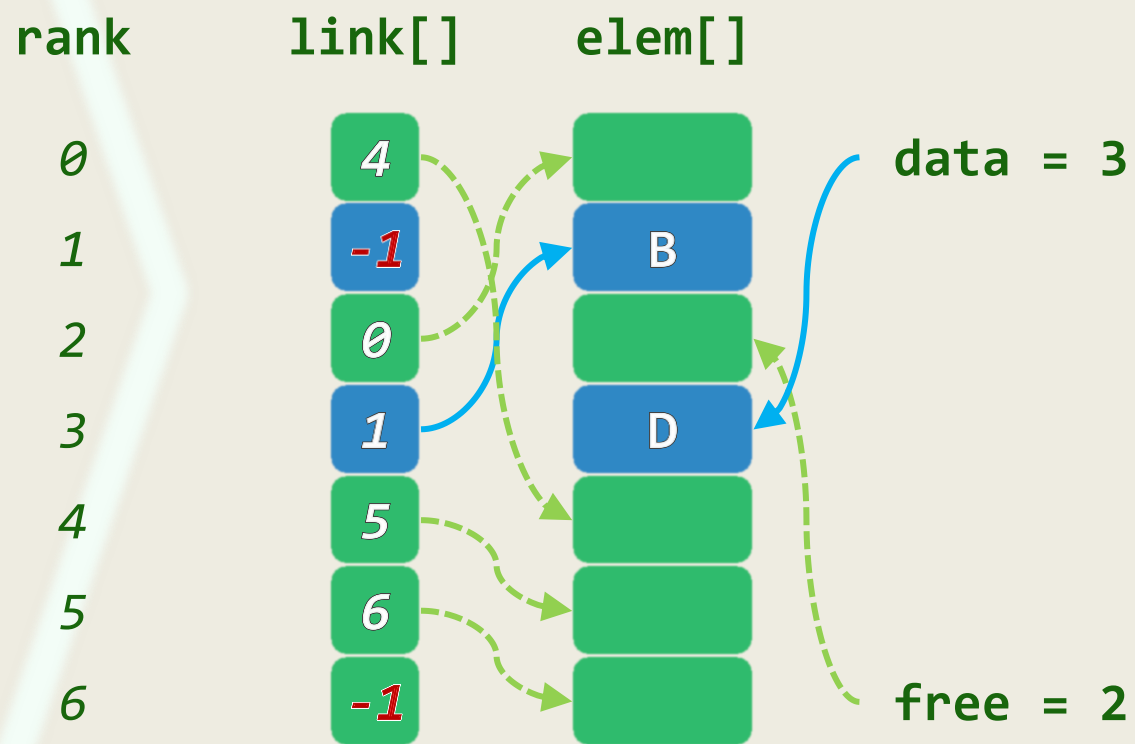


实例 (3/4)

remove('A')

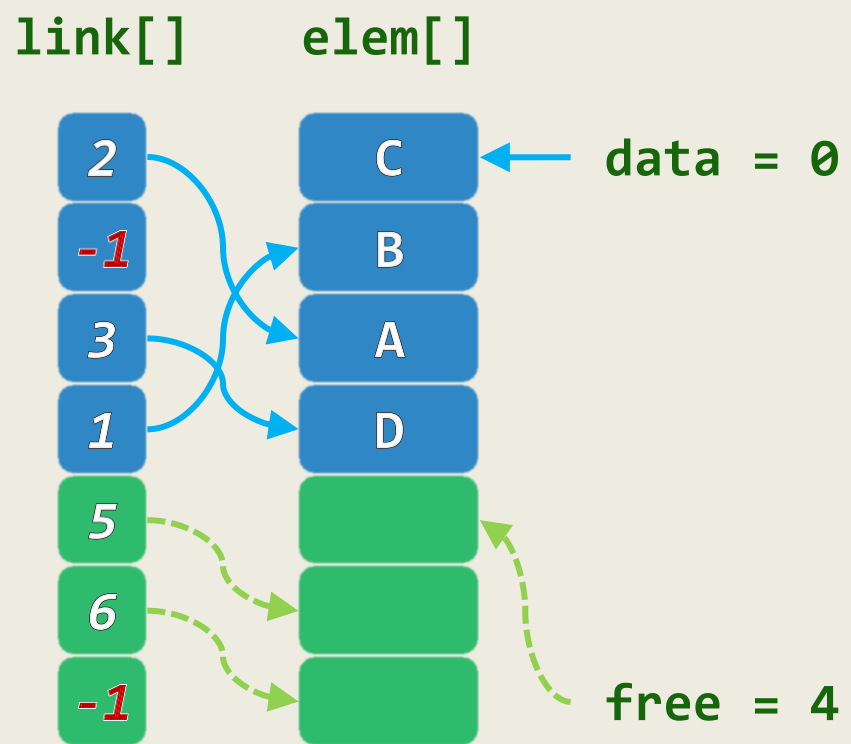


remove('C')



实例 (4/4)

insert('C')



rank

0

1

2

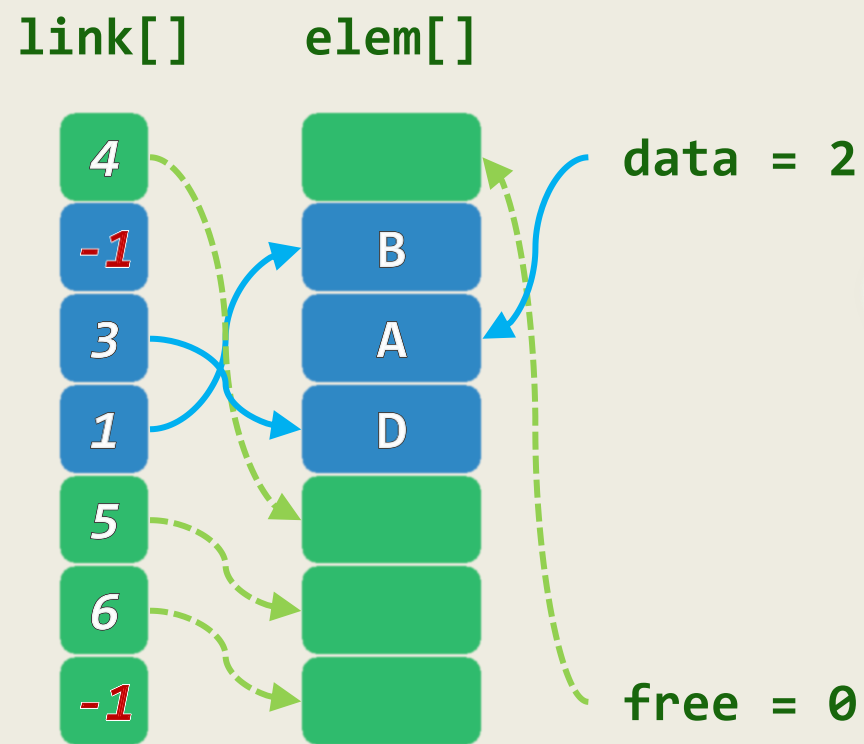
3

4

5

6

insert('A')



03-XB

列表

Java序列

One of the things that Java is good at is giving you this
homogeneous view of a reality that's usually very heterogeneous.

邓俊辉

deng@tsinghua.edu.cn

Interface: 定义

❖ Java支持ADT的一种机制：在同一接口规范下，允许不同的实现

❖ **interface** Geometry { //几何物体

final double PI = 3.1415926; //常量定义，类定义可直接使用

double area(); //无参数的接口方法

boolean inside(**Point** p); //带参数的接口方法

}

❖ **interface**不能直接实例化为对象

符合**interface**定义的任何类，都需要具体地**实现**其中的接口方法

Interface: 实现

```
class Disk implements Geometry { //符合Geometry接口的Disk类
    Point c; double r;

    public Disk( Point center, double radius ) //构造方法
    {c = center; r = radius;}

    public double perimeter() { return 2 * PI * r; } //类方法
    public double area() { return PI * r * r; } //接口实现
    public boolean inside( Point p ) { //接口实现
        double dx = p.x - c.x, dy = p.y - c.y;
        return dx*dx + dy*dy < r*r;
    }
}
```

向量接口: Vector.java

```
public interface Vector {  
    public int getSize();  
    public boolean isEmpty();  
    public Object getAtRank( int r ) throws ExceptionBoundaryViolation;  
    public Object replaceAtRank( int r, Object obj )  
        throws ExceptionBoundaryViolation;  
    public Object insertAtRank( int r, Object obj )  
        throws ExceptionBoundaryViolation;  
    public Object removeAtRank( int r ) throws ExceptionBoundaryViolation;  
}
```

向量实现1: Vector_Array.java

```
public class Vector_Array implements Vector {  
    private final int N = 1024; //数组容量固定  
    private Object[] A; private int n = 0;  
    public Vector_Array() { A = new Object[N]; n = 0; }  
    public int getSize() { return n; }  
    public boolean isEmpty() { return 0 == n; }  
    public Object insertAtRank( int r, Object obj ) throws ExceptionBoundaryViolation {  
        if ( 0 > r || r > n ) throw new ExceptionBoundaryViolation( "out of range" );  
        if ( n >= N ) throw new ExceptionBoundaryViolation( "overflow" );  
        for ( int i = n; i > r; i-- ) A[i] = A[i - 1];  
        A[r] = obj; n++; return obj;  
    }  
    /* ..... */  
}
```

向量实现2: Vector_ExtArray.java

```
public class Vector_ExtArray implements Vector {  
    private int N = 8; //数组的初始容量, 可不断增加  
    /* ..... */  
    public Object insertAtRank( int r, Object obj ) throws ExceptionBoundaryViolation {  
        if ( 0 > r || r > n ) throw new ExceptionBoundaryViolation( "out of range" );  
        if ( N <= n ) { //空间溢出的处理  
            N *= 2; Object B[] = new Object[ N ]; //容量加倍  
            for ( int i = 0; i < n; i++ ) B[i] = A[i]; A = B; //用B[]替换A[]  
        }  
        for ( int i = n; i > r; i-- ) A[i] = A[i - 1]; //后续元素顺次后移  
        A[r] = obj;  n++;  return obj;  
    }  
    /* ..... */  
}
```

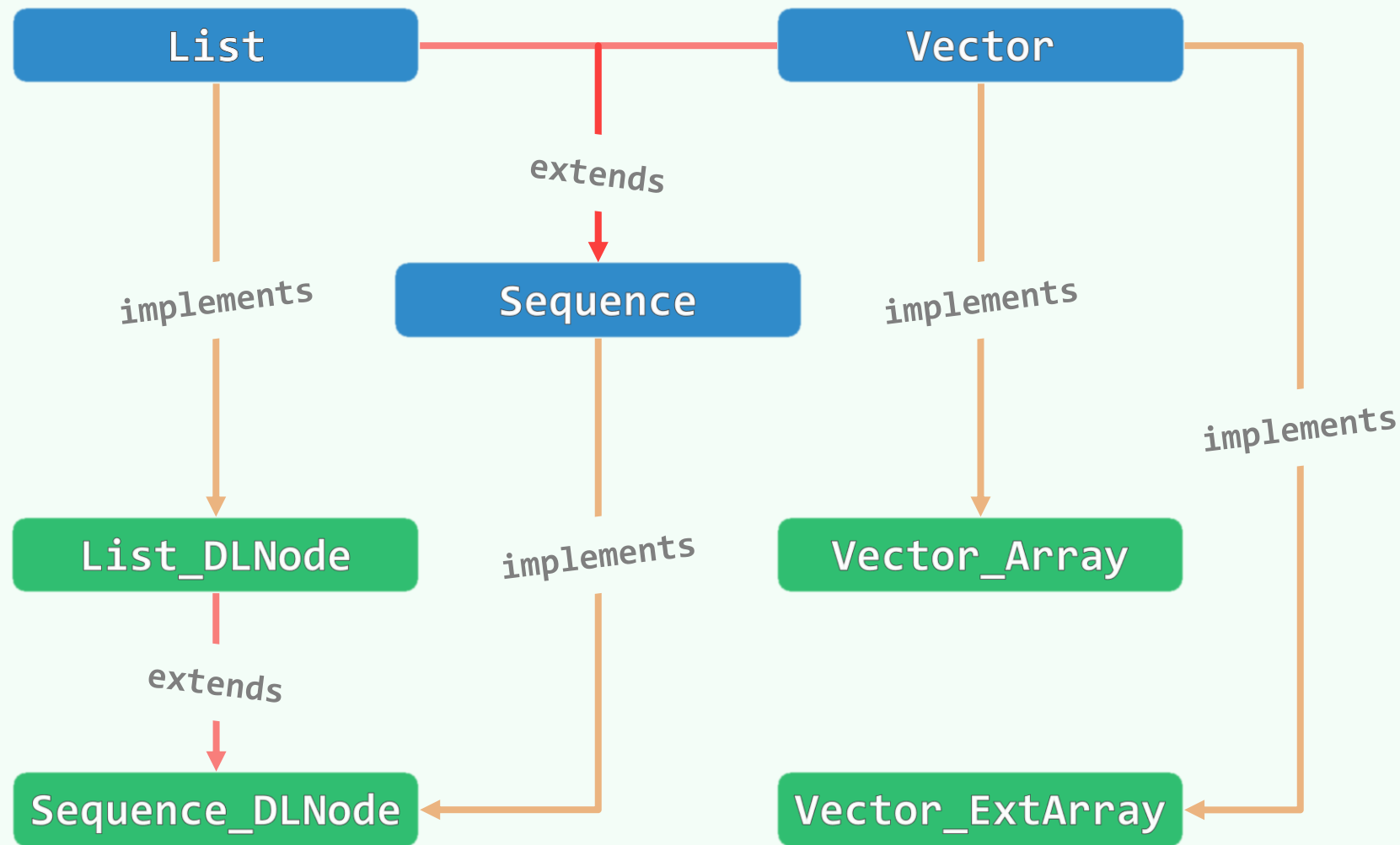
序列接口及其实现

```
❖ interface List
{ /* ... */ }

class List_DLNode
implements List
{ /* ... */ }

❖ interface Sequence
extends Vector, List
{ /* ... */ }

class Sequence_DLNode
extends List_DLNode
implements Sequence
{ /* ... */ }
```



03-XC

列表

Python列表

邓俊辉

PYTHON = Programmers Yearning To Homestead Our Noosphere.

deng@tsinghua.edu.cn

声明 + 倒置 + 排序

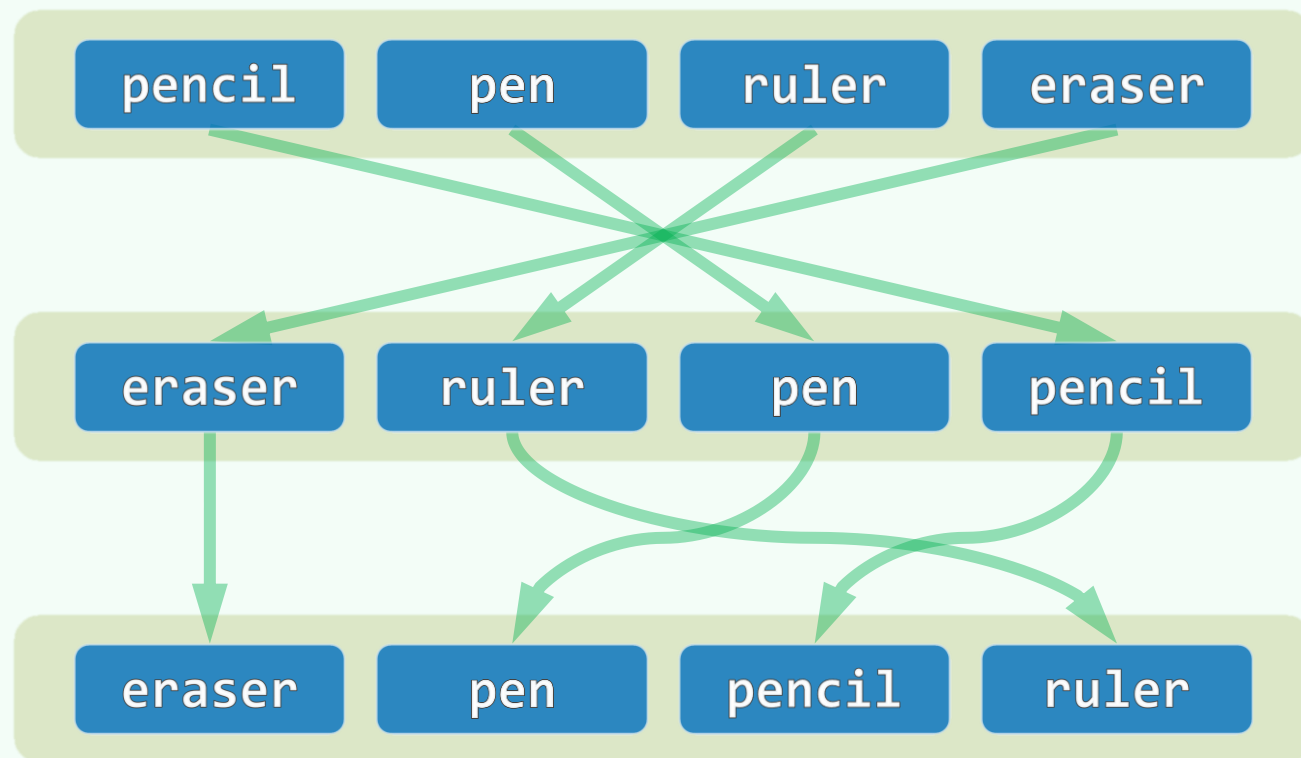
❖ 在Python中, List属于内置的标准数据类型

❖ `box = [ 'pencil', 'pen', 'ruler', 'eraser' ]; print(box)`
['pencil', 'pen', 'ruler', 'eraser']

❖ `for item in box: print(item),`
pencil pen ruler eraser

❖ `box.reverse()`
`for item in box: print(item),`
eraser ruler pen pencil

❖ `box.sort()`
`for item in box: print(item),`
eraser pen pencil ruler



区间遍历

❖ `for i in range(0, len(box)): # [0, n)`

`print(box[i]),`

`# eraser pen pencil ruler`

❖ `for i in range(len(box)-1, -1, -1): # [n-1, -1)`

`print(box[i]),`

`# ruler pencil pen eraser`

❖ `for i in range(-1, -len(box)-1, -1): # [-1, -n-1)`

`print(box[i]),`

`# ruler pencil pen eraser`

eraser

pen

pencil

ruler

集合遍历

❖ `bag = [ 'data structures', 'calculus', box, 2012012012 ]`

`print(bag)`

`# ['data structures', 'calculus',
['eraser', 'pen', 'pencil', 'ruler'], 2012012012]`

❖ `for item in bag: print(item),`

`# data structures calculus`

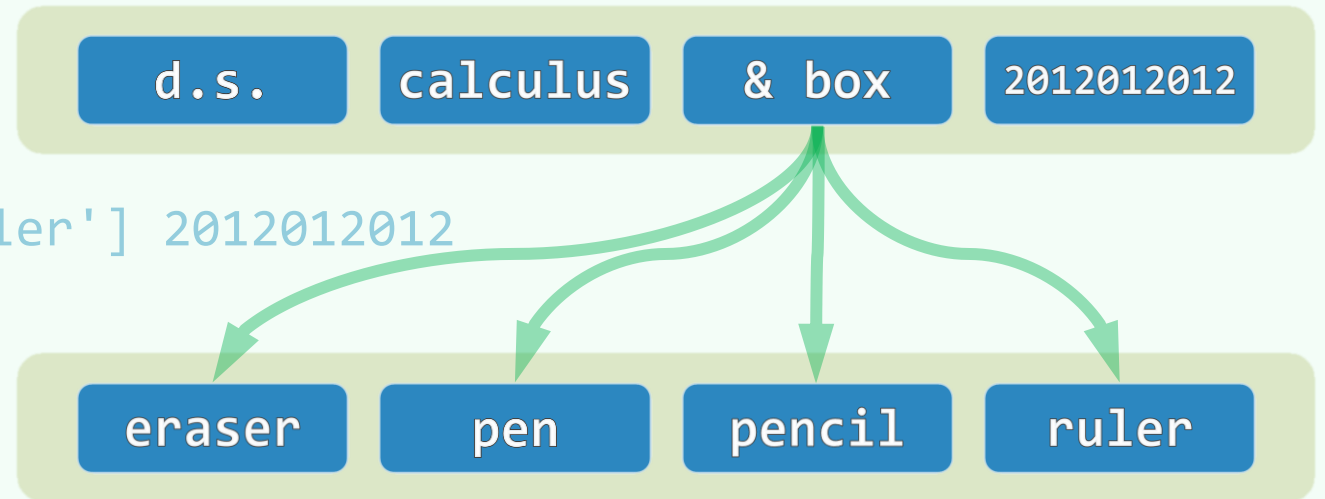
`['eraser', 'pen', 'pencil', 'ruler'] 2012012012`

❖ `for item in bag[2]: print(item),`

`# eraser pen pencil ruler`

❖ `for item in bag[2][1:3]: print(item),`

`# pen pencil`



reverse(): 秩 + 位置

```
def reverse_1(L): # 循秩访问?
```

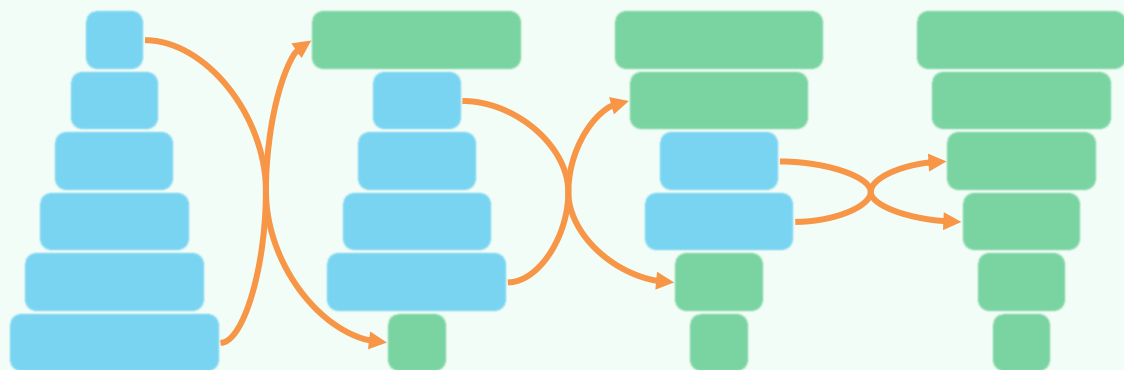
```
    lo, hi = 0, len(L) - 1
```

```
    while lo < hi:
```

```
        L[lo], L[hi] = L[hi], L[lo]
```

```
        lo, hi = lo + 1, hi - 1
```

```
    return L
```



```
def reverse_2(L): # 循位置访问?
```

```
    for i in range( len(L) ):
```

```
        L.insert(i, L.pop())
```

```
    return L
```

